Cours PHP

Créer un site web dynamique

Niveau : débutant

Site dynamique · transmission de données · bases de données · projet final

Octobre 2026

Sommaire

Présentation de la formation 6
Objectifs 6
Public et prérequis 6
Organisation 6
Conventions des exemples 6
Partie 1 — Créer un site web dynamique 7
Chapitre P1 — Le principe d'un site web dynamique 8
P1.1 Page statique et page dynamique 8
P1.2 Le trajet d'une requête 8
Synthèse 8
Quiz 9
Exercices 9
Corrigés 9
Chapitre P2 — L'environnement de travail 11
P2.1 Les outils 11
P2.2 Le dossier htdocs et l'adresse localhost 11
P2.3 Une première page et une première erreur 12
Synthèse 13
Quiz 13
Exercices 13
Corrigés 14
Chapitre P3 — La structure d'un script et l'affichage 15
P3.1 La structure de base 15
P3.2 Afficher avec echo 15
P3.3 Guillemets doubles ou simples 16
Synthèse 16
Quiz 16
Exercices 17
Corrigés 17
Chapitre P4 — Variables et types 19
P4.1 Les variables 19
P4.2 Opérateurs 19
P4.3 Le transtypage 20
P4.4 Vérifier qu'une variable existe : isset 21
Synthèse 21
Quiz 21
Exercices 21
Corrigés 22
Chapitre P5 — Les chaînes de caractères 23
P5.1 Les fonctions de l'annexe 23
P5.2 Exemple : générer un identifiant 24
Synthèse 24
Quiz 24
Exercices 25

Corrigés 25
Chapitre P6 — Les tableaux indicés et associatifs 27
P6.1 Le tableau indicé 27
P6.2 Le tableau associatif 27
P6.3 Parcourir un tableau 27
Synthèse 29
Quiz 30
Exercices 30
Corrigés 30
Chapitre P7 — Les structures de contrôle 32
P7.1 Opérateurs de comparaison et logiques 32
P7.2 La structure if 32
P7.3 Les boucles for, while et do ... while 33
P7.4 Les fonctions mathématiques 35
Synthèse 35
Quiz 35
Exercices 36
Corrigés 36
Chapitre P8 — La date et l'heure 38
P8.1 Le fuseau horaire 38
P8.2 Formater une date avec date() 38
P8.3 Timestamp, écart entre dates et validité 38
Synthèse 39
Quiz 40
Exercices 40
Corrigés 40
Chapitre P9 — Importer un fichier avec require 42
P9.1 Le principe 42
P9.2 Exemple : un en-tête commun 42
Synthèse 43
Quiz 43
Exercices 44
Corrigés 44
Partie 2 — Transmettre des données entre pages 45
Chapitre P10 — Le principe de la transmission 46
P10.1 Deux méthodes HTTP 46
P10.2 Les paramètres d'URL 46
P10.3 Les variables superglobales 46
Synthèse 48
Quiz 48
Exercices 48
Corrigés 48
Chapitre P11 — Transmettre par l'URL avec $_GET 50
P11.1 Exemple : du catalogue à la fiche 50
Synthèse 52
Quiz 52
Exercices 52



Corrigés ... 52
Chapitre P12 — Transmettre par un formulaire avec $_POST ... 54
P12.1 Le formulaire et sa page de traitement ... 54
P12.2 Exemple : l'inscription à la bibliothèque ... 54
P12.3 GET ou POST ? ... 55
Synthèse ... 56
Quiz ... 56
Exercices ... 56
Corrigés ... 57
Partie 3 — Interagir avec une base de données ... 58
Chapitre P13 — La base de données dans phpMyAdmin ... 59
P13.1 Rappels ... 59
P13.2 La base « bibliotheque » ... 59
P13.3 Les requêtes générées en mode assisté ... 60
Synthèse ... 62
Quiz ... 62
Exercices ... 63
Corrigés ... 63
Chapitre P14 — Se connecter et sélectionner la base ... 64
P14.1 mysqli_connect ... 64
P14.2 Le fichier connexion.php ... 64
Synthèse ... 65
Quiz ... 65
Exercices ... 66
Corrigés ... 66
Chapitre P15 — Afficher des données : SELECT ... 67
P15.1 Les étapes ... 67
P15.2 Exemple : le catalogue ... 68
P15.3 Exemple : la fiche d'un livre ... 69
Synthèse ... 69
Quiz ... 69
Exercices ... 70
Corrigés ... 70
Chapitre P16 — Rechercher avec un formulaire ... 72
P16.1 Le principe ... 72
P16.2 Exemple : rechercher par genre et par auteur ... 72
Synthèse ... 74
Quiz ... 74
Exercices ... 74
Corrigés ... 75
Chapitre P17 — Ajouter des données : INSERT ... 76
P17.1 La requête INSERT ... 76
P17.2 Exemple : le formulaire d'ajout ... 76
P17.3 Le piège de l'apostrophe ... 78
Synthèse ... 79
Quiz ... 79
Exercices ... 80



Corrigés ... 80
Chapitre P18 — Supprimer des données : DELETE ... 82
P18.1 La requête DELETE ... 82
P18.2 La liste de gestion ... 82
P18.3 La page de suppression avec confirmation ... 83
Synthèse ... 84
Quiz ... 84
Exercices ... 85
Corrigés ... 85
Chapitre P19 — Modifier des données : UPDATE ... 87
P19.1 La requête UPDATE ... 87
P19.2 Le formulaire pré-rempli ... 87
Synthèse ... 89
Quiz ... 89
Exercices ... 89
Corrigés ... 90
Chapitre P20 — Une application complète ... 91
P20.1 L'organisation des fichiers ... 91
P20.2 Récapitulatif ... 91
P20.3 Le parcours complet ... 91
Synthèse ... 92
Quiz ... 93
Exercices ... 93
Corrigés ... 93
Projet final — Le site dynamique « Centre Numérique Avenir » ... 94
La base de données ... 94
Cahier des charges ... 94
Grille d'évaluation sur 20 ... 94
Éléments de correction : inscription.php ... 95
Annexe — Mémo PHP ... 96
Types, opérateurs et affichage ... 96
Fonctions ... 96
MySQLi, interface procédurale ... 96
Requêtes SQL ... 97

Présentation de la formation

Objectifs

À la fin de ce cours, vous serez capable de :

- expliquer le fonctionnement d'un site web dynamique et installer un environnement de travail ;
- écrire des scripts PHP manipulant variables, chaînes, tableaux, dates et structures de contrôle ;
- réutiliser du code avec require ;
- transmettre des données entre des pages avec $_GET et $_POST ;
- se connecter à une base de données MySQL, l'interroger et la mettre à jour (SELECT, INSERT, DELETE, UPDATE) depuis des formulaires ;
- intégrer les résultats d'une requête dans une page web.

Public et prérequis

- Public : apprenants ayant suivi les cours HTML5, CSS3 et JavaScript.
- Prérequis : savoir écrire une page HTML avec un formulaire ; notions de base de données (table, champ, enregistrement).
- Matériel : XAMPP (Apache, MySQL, phpMyAdmin), Visual Studio Code, un navigateur récent.

Organisation

- **Partie 1 — Créer un site web dynamique** : principe, environnement, syntaxe de PHP, chaînes, tableaux, dates, structures de contrôle, require.
- **Partie 2 — Transmettre des données entre pages** : $_GET par l'URL, $_POST par un formulaire.
- **Partie 3 — Interagir avec une base de données** : la partie principale du cours. Connexion, puis requêtes SELECT, INSERT, DELETE et UPDATE pilotées par des formulaires, jusqu'à une application complète.
- **Projet final** : rendre dynamique le site « Centre Numérique Avenir » réalisé dans les cours précédents.

Conventions des exemples

Chaque exemple présente le ou les fichiers PHP, puis une capture du navigateur montrant la page réellement exécutée par le serveur. Les exemples de la partie 3 s'appuient sur la base « bibliotheque » ; les exercices, sur la base « boutique ». Le script de création des deux bases est fourni au chapitre P13.

Partie 1 — Créer un site web dynamique

Cette partie présente le principe d'un site dynamique, l'environnement de travail et les bases du langage PHP.

Chapitre P1 — Le principe d'un site web dynamique

OBJECTIFS DU CHAPITRE

- Distinguer une page statique d'une page dynamique.
- Décrire le trajet d'une requête vers une page PHP.
- Citer le rôle du serveur web, de l'interpréteur PHP et du SGBD.

P1.1 Page statique et page dynamique

	Page statique	Page dynamique
Fichier	index.html	index.php
Contenu	Identique à chaque visite	Calculé à chaque visite (date, données d'une base, saisie de l'utilisateur)
Traitement	Le serveur envoie le fichier tel quel	Le serveur exécute le script, puis envoie le HTML produit
Exemple	Page « Qui sommes-nous ? »	Catalogue lu dans une base, résultat d'une recherche

P1.2 Le trajet d'une requête

1. Le navigateur (client) demande la page http://localhost/biblio/catalogue.php.
2. Le serveur web (Apache) reçoit la demande et voit l'extension .php.
3. Il confie le fichier à l'interpréteur PHP, qui exécute le script.
4. Si besoin, le script interroge le SGBD (MySQL) et reçoit des données.
5. Le script produit du HTML ; le serveur l'envoie au navigateur.
6. Le navigateur affiche la page. Il ne reçoit jamais le code PHP.

À RETENIR

PHP s'exécute sur le serveur, JavaScript dans le navigateur. Dans le code source de la page (Ctrl + U), on ne voit que le HTML produit, jamais le code PHP.

Synthèse

À RETENIR

- Statique : envoyé tel quel. Dynamique : calculé à chaque visite.
- Trajet : navigateur → Apache → PHP (→ MySQL) → HTML → navigateur.
- Le code PHP n'est jamais envoyé au navigateur.

Quiz

1. Où s'exécute un script PHP ?

a) Dans le navigateur
b) Sur le serveur
c) Dans la base de données
d) Dans l'éditeur

2. Que reçoit le navigateur quand il demande une page PHP ?

a) Le code PHP
b) Du HTML
c) La base de données
d) Un fichier .php à télécharger

3. Quel logiciel stocke les données ?

a) Apache
b) Le navigateur
c) Le SGBD MySQL
d) VS Code

Exercices

Exercice 1 — Statique ou dynamique ?

Pour chaque page, dites si elle doit être statique ou dynamique : a) les mentions légales d'un site ; b) la liste des livres disponibles d'une bibliothèque ; c) le résultat d'une recherche ; d) un plan d'accès ; e) le panier d'un site de vente.

Exercice 2 — Le trajet d'une requête

Remettez dans l'ordre : (1) PHP exécute le script ; (2) le navigateur affiche la page ; (3) le navigateur demande catalogue.php ; (4) MySQL renvoie les livres ; (5) Apache reçoit la demande ; (6) le HTML est envoyé au navigateur.

Corrigés

Quiz

1. b) L'interpréteur PHP fonctionne sur le serveur.

2. b) Seul le HTML produit est envoyé.

3. c) Le SGBD gère la base de données.

Exercice 1 — Statique ou dynamique ?

a) statique ; b) dynamique (lue dans une base) ; c) dynamique (dépend de la saisie) ; d) statique ; e) dynamique (propre à chaque visiteur).

Exercice 2 — Le trajet d'une requête

3 → 5 → 1 → 4 → 6 → 2.

Chapitre P2 — L'environnement de travail

OBJECTIFS DU CHAPITRE

- Installer et démarrer XAMPP.
- Placer ses fichiers dans le dossier htdocs.
- Ouvrir une page avec l'adresse http://localhost.
- Lire un message d'erreur PHP.

P2.1 Les outils

Outil	Rôle
XAMPP	Ensemble prêt à l'emploi : serveur web Apache, interpréteur PHP, SGBD MySQL (MariaDB) et phpMyAdmin
Panneau de contrôle XAMPP	Démarre et arrête Apache et MySQL (boutons Start / Stop)
phpMyAdmin	Interface web de gestion des bases : http://localhost/phpmyadmin
Visual Studio Code	Éditeur des fichiers .php
Navigateur	Affiche les pages produites par le serveur

P2.2 Le dossier htdocs et l'adresse localhost

Le serveur ne publie que les fichiers placés dans le dossier `C:\xampp\htdocs`. Créez un dossier par projet, puis ouvrez la page avec l'adresse `http://localhost/dossier/fichier.php`.

```
C:\xampp\htdocs\
└── biblio\
    ├── index.php      →  http://localhost/biblio/index.php
    └── catalogue.php  →  http://localhost/biblio/catalogue.php
```

ATTENTION

Ne jamais ouvrir un fichier .php par un double-clic : le navigateur l'afficherait sans l'exécuter. Passez toujours par http://localhost, avec Apache démarré.

P2.3 Une première page et une première erreur

index.php

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Première page</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
<h1>Bibliothèque municipale</h1>
<?php
echo "<p>Bienvenue sur la page dynamique de la bibliothèque.</p>";
?>
</body>
</html>
```

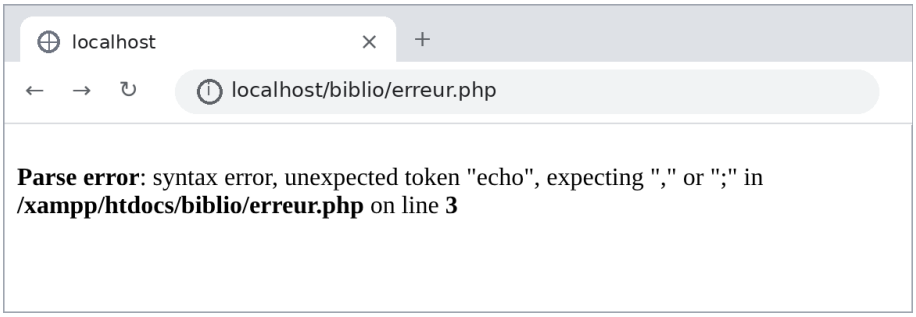


Résultat dans le navigateur

Quand le script contient une erreur, PHP affiche un message avec le type d'erreur, le fichier et le numéro de la ligne. Ici, un point-virgule manque à la fin de la ligne 2 : PHP signale la ligne 3, là où il rencontre l'instruction inattendue. Sous Windows, le chemin affiché commence par C:\xampp\htdocs.

erreur.php

```
<?php
echo "Bonjour"
echo "à tous";
?>
```



Message d'erreur de PHP : type, fichier et numéro de ligne

Synthèse

À RETENIR

- XAMPP : Apache + PHP + MySQL + phpMyAdmin.
- Fichiers dans C:\xampp\htdocs\dossier ; page ouverte par http://localhost/dossier/page.php.
- Une erreur PHP indique le type, le fichier et la ligne.

Quiz

1. Où placer les fichiers d'un site avec XAMPP ?

a) Sur le bureau
b) Dans C:\xampp\htdocs
c) Dans Mes documents
d) N'importe où

2. Quelle adresse ouvre le fichier htdocs\biblio\index.php ?

a) file:///index.php
b) http://localhost/biblio/index.php
c) http://biblio/index
d) C:\biblio\index.php

3. Que faut-il démarrer dans le panneau XAMPP pour afficher une page PHP ?

a) MySQL seulement
b) Apache
c) phpMyAdmin
d) Rien

Exercices

Exercice 1 — Ma première page PHP

Créez le dossier htdocs\essai et la page index.php qui affiche, avec echo, le titre « Mon premier site dynamique » et un paragraphe avec votre prénom. Ouvrez-la avec http://localhost/essai/index.php, puis affichez le code source (Ctrl + U).

Exercice 2 — Où placer le fichier ?

Un apprenant a enregistré contact.php sur son bureau et l'ouvre par un double-clic : le navigateur affiche le code au lieu de la page. Expliquez le problème et donnez la marche à suivre.

Corrigés

Quiz

1. b) Apache ne publie que le dossier htdocs.

2. b) localhost désigne le serveur local.

3. b) Apache sert les pages et les confie à PHP.

Exercice 1 — Ma première page PHP

```
<h1>Mon premier site dynamique</h1>
<?php
echo "<p>Page créée par Yasmine.</p>";
?>
```

Dans le code source, on voit uniquement le HTML produit : la balise <?php et l'instruction echo n'apparaissent pas.

Exercice 2 — Où placer le fichier ?

Le fichier n'est pas exécuté par le serveur. Il faut démarrer Apache, déplacer contact.php dans un dossier de C:\xampp\htdocs (par exemple htdocs\site) et l'ouvrir avec http://localhost/site/contact.php.

Chapitre P3 — La structure d'un script et l'affichage

OBJECTIFS DU CHAPITRE

- Identifier la structure de base d'un script PHP.
- Afficher du texte et du HTML avec echo.
- Concaténer avec l'opérateur point.
- Distinguer guillemets simples et doubles.

P3.1 La structure de base

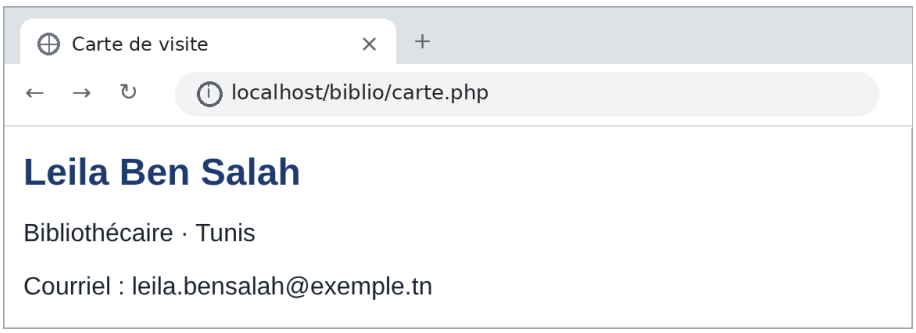
- `<?php` ouvre un bloc de code PHP, `?>` le ferme. Le reste du fichier est du HTML envoyé tel quel.
- Chaque instruction se termine par un point-virgule.
- **Commentaires** : `// sur une ligne` ou `/* sur plusieurs lignes */`.
- Le fichier porte l'extension .php, même s'il contient surtout du HTML.

P3.2 Afficher avec echo

echo affiche une ou plusieurs expressions, séparées par des virgules. L'opérateur `.` (point) concatène des chaînes. echo peut produire du HTML.

carte.php

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Carte de visite</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
<?php
// Carte de visite générée par PHP
$nom = "Leila Ben Salah";
$metier = "Bibliothécaire";
echo "<h1>" . $nom . "</h1>";
echo "<p>", $metier, " · Tunis</p>";
echo '<p>Courriel : leila.bensalah@exemple.tn</p>';
?>
</body>
</html>
```



Résultat dans le navigateur

P3.3 Guillemets doubles ou simples

Écriture	Résultat	Explication
echo "Bonjour $nom";	Bonjour Leila	Entre guillemets doubles, la variable est remplacée par sa valeur
echo 'Bonjour $nom';	Bonjour $nom	Entre guillemets simples, le texte est affiché tel quel
echo 'Bonjour ' . $nom;	Bonjour Leila	Concaténation : fonctionne dans les deux cas

ASTUCE

Pour produire du HTML contenant des guillemets doubles (attributs), utilisez des guillemets simples autour de la chaîne : echo 'Accueil';

Synthèse

À RETENIR

- <?php … ?> : bloc PHP ; ; à la fin de chaque instruction ; // et /* */.
- echo affiche texte et HTML ; . concatène.
- Guillemets doubles : variables remplacées ; simples : texte brut.

Quiz

1. Quel est l'opérateur de concaténation en PHP ?

a) +
b) .
c) &
d) ,

2. $a = "Sfax"; echo 'Ville : $a'; affiche :

a) Ville : Sfax

b) Ville : $a

c) Une erreur

d) Rien

3. Quelle balise ouvre un bloc PHP ?

a) <php>

b) <?php

c) <script php>

d) <%

Exercices

Exercice 1 — Ma carte de visite

Créez carte.php. Rangez votre nom, votre métier et votre ville dans trois variables, puis affichez une carte de visite : le nom en <h1>, le métier et la ville dans un paragraphe, en utilisant la concaténation.

Exercice 2 — Trouver les erreurs

Ce script contient trois erreurs. Corrigez-les.

```
<?
$titre = "Catalogue"
echo "<h1>" + $titre + "</h1>";
?>
```

Corrigés

Quiz

1. b) Le point concatène les chaînes.

2. b) Les guillemets simples n'interprètent pas les variables.

3. b) <?php … ?>

Exercice 1 — Ma carte de visite

```
<?php
$nom = "Karim Trabelsi";
$metier = "Technicien réseaux";
$ville = "Sfax";
echo "<h1>" . $nom . "</h1>";
echo "<p>" . $metier . " à " . $ville . "</p>";
?>
```

Exercice 2 — Trouver les erreurs

```
<?php
$titre = "Catalogue";
echo "<h1>" . $titre . "</h1>";
?>
```

Balise d'ouverture incomplète (<?php), point-virgule manquant, + au lieu de . pour concaténer.

Chapitre P4 — Variables et types

OBJECTIFS DU CHAPITRE

- Déclarer des variables de types entier, réel, booléen et chaîne.
- Calculer avec les opérateurs arithmétiques.
- Convertir un type avec les opérateurs de transtypage.
- Vérifier qu'une variable existe avec isset.

P4.1 Les variables

Une variable commence par `$`, suivi d'un nom (lettre ou _ au début, sans espace ni accent). Elle est créée par sa première affectation avec `=` ; son type dépend de la valeur. La casse compte : $prix et $Prix sont deux variables.

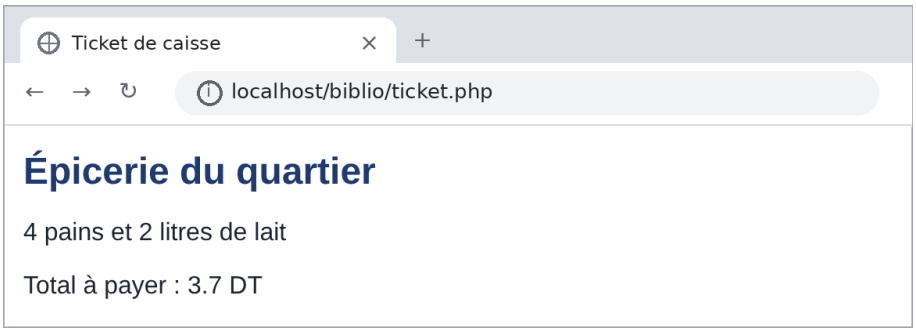
Type	Exemple	Remarque
int (entier)	$quantite = 3;	
float (réel)	$prix = 2.5;	Séparateur décimal : le point
string (chaîne)	$ville = "Tunis";	Entre guillemets
bool (booléen)	$disponible = true;	true ou false
array (tableau)	$notes = [12, 15];	Chapitre P6

P4.2 Opérateurs

Opérateur	Rôle	Exemple	Résultat
+ - * /	Addition, soustraction, multiplication, division	7 / 2	3.5
%	Modulo (reste de la division entière)	7 % 2	1
=	Affectation	$a = 5;	$a vaut 5
.	Concaténation	"Bon" . "jour"	"Bonjour"

ticket.php

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Ticket de caisse</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
<?php
$prixPain = 0.250;
$prixLait = 1.350;
$total = $prixPain * 4 + $prixLait * 2;
echo "<h1>Épicerie du quartier</h1>";
echo "<p>4 pains et 2 litres de lait</p>";
echo "<p>Total à payer : " . $total . " DT</p>";
?>
</body>
</html>
```



Résultat dans le navigateur

P4.3 Le transtypage

Opérateur	Exemple	Résultat
(int)	(int) "12.7 kg"	12
(float)	(float) "12.7 kg"	12.7
(string)	(string) 40	"40"
(bool)	(bool) 0	false
(array)	(array) "Tunis"	["Tunis"]

À RETENIR

Une donnée reçue d'un formulaire ou d'une URL est une chaîne. Le transtypage (int) est très utile pour un identifiant : (int) "3" vaut 3, et une valeur non numérique donne 0.

P4.4 Vérifier qu'une variable existe : isset

isset($variable) retourne true si la variable existe et n'est pas nulle. Il servira surtout à vérifier qu'une donnée a bien été transmise (partie 2).

Synthèse

> **À RETENIR**
>
> - $nom = valeur; : le type suit la valeur (int, float, string, bool, array).
> - + - * / % ; = affecte ; . concatène.
> - (int) (float) (string) (bool) (array) convertissent ; isset teste l'existence.

Quiz

1. Quel nom de variable est valide ?

a) $2prix
b) $prix total
c) $prixTotal
d) prixTotal

2. Que vaut (int) "45 DT" ?

a) 45
b) 0
c) "45"
d) Une erreur

3. Que vaut 17 % 5 ?

a) 3.4
b) 2
c) 3
d) 12

Exercices

Exercice 1 — Le ticket de caisse

Au café, un client prend 2 cafés à 1.200 DT et 3 croissants à 0.800 DT. Rangez prix et quantités dans des variables, calculez le total et affichez « Montant : … DT ».

Exercice 2 — Le transtypage

Donnez la valeur et le type de chaque expression : a) (int) "7.9" ; b) (float) "3,5" ; c) (bool) "" ; d) (string) 2.50 ; e) (int) "abc".

Corrigés

Quiz

1. c) $ puis une lettre, sans espace.

2. a) Le début numérique est conservé.

3. b) 17 = 5 × 3 + 2.

Exercice 1 — Le ticket de caisse

```
<?php
$prixCafe = 1.2;
$prixCroissant = 0.8;
$total = $prixCafe * 2 + $prixCroissant * 3;
echo "<p>Montant : " . $total . " DT</p>";   // 4.8
?>
```

Exercice 2 — Le transtypage

a) 7 (int) ; b) 3 (float : la virgule n'est pas un séparateur décimal) ; c) false (bool) ; d) "2.5" (string) ; e) 0 (int).

Chapitre P5 — Les chaînes de caractères

OBJECTIFS DU CHAPITRE

- Obtenir la longueur et extraire une partie d'une chaîne.
- Rechercher, comparer et remplacer.
- Changer la casse et supprimer les espaces inutiles.
- Passer d'un caractère à son code ASCII.

P5.1 Les fonctions de l'annexe

Les positions des caractères commencent à 0.

Fonction	Rôle	Exemple avec $ch = "Carthage"	Résultat
strlen($ch)	Longueur	strlen($ch)	8
substr($ch, d, n)	Sous-chaîne de n caractères à partir de d	substr($ch, 0, 4)	"Cart"
strpos($ch, $m)	Position de la 1re occurrence de $m (false si absent)	strpos($ch, "a")	1
strcmp($a, $b)	Compare : < 0, 0 ou > 0	strcmp("abc", "abd")	< 0
str_replace($m, $r, $ch)	Remplace toutes les occurrences de $m par $r	str_replace("a", "A", $ch)	"CArthAge"
strtolower($ch) · strtoupper($ch)	Minuscules · majuscules	strtoupper($ch)	"CARTHAGE"
trim($ch)	Supprime les espaces au début et à la fin	trim(" Tunis ")	"Tunis"
chr($code) · ord($c)	Caractère d'un code ASCII · code du 1er caractère	chr(65) · ord("a")	"A" · 97

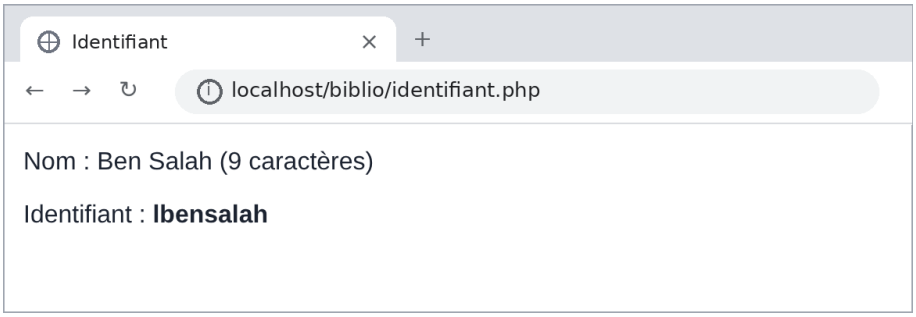
ATTENTION

substr prend une longueur (n caractères), pas une position de fin. str_replace remplace toutes les occurrences, et l'ordre de ses paramètres est : motif, remplacement, chaîne.

P5.2 Exemple : générer un identifiant

identifiant.php

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Identifiant</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
<?php
$prenom = "  Leila ";
$nom = "Ben Salah";
// initiale du prénom + nom sans espace, en minuscules
$login = substr(trim($prenom), 0, 1) . str_replace(" ", "", $nom);
$login = strtolower($login);
echo "<p>Nom : $nom (" . strlen($nom) . " caractères)</p>";
echo "<p>Identifiant : <b>$login</b></p>";
?>
</body>
</html>
```



Résultat dans le navigateur

Synthèse

À RETENIR

- strlen, substr(d, n), strpos (false si absent), strcmp (0 si égales).
- str_replace : toutes les occurrences ; strtolower, strtoupper, trim.
- chr : code → caractère ; ord : caractère → code.

Quiz

1. Que vaut substr("Bizerte", 1, 3) ?

a) "Biz"

b) "ize"

c) "izer"

d) "iz"

2. Que retourne strcmp("Tunis", "Tunis") ?

a) true

b) 0

c) 1

d) -1

3. Que vaut ord("A") ?

a) 1

b) 65

c) "A"

d) 97

Exercices

Exercice 1 — Générateur d'identifiant

À partir de $prenom = "Amel" et $nom = "Gharbi", construisez l'identifiant : les trois premières lettres du prénom suivies du nom, en minuscules, puis ajoutez « @biblio.tn ». Affichez l'adresse obtenue et sa longueur.

Exercice 2 — Vérifier un mot de passe

Un mot de passe est accepté s'il contient au moins 8 caractères et ne contient pas le mot « 1234 ». Testez $mdp = "biblio1234" et $mdp = "Carthage2026", et affichez « Accepté » ou « Refusé » avec la raison. (La structure if est révisée au chapitre P7.)

Corrigés

Quiz

1. b) 3 caractères à partir de la position 1.

2. b) 0 : chaînes identiques.

3. b) Code ASCII de A.

Exercice 1 — Générateur d'identifiant

```
<?php
$prenom = "Amel";
$nom = "Gharbi";
$adresse = strtolower(substr($prenom, 0, 3) . $nom) . "@biblio.tn";
echo $adresse . " (" . strlen($adresse) . " caractères)";   //
amegharbi@biblio.tn (19 caractères)
?>
```

Exercice 2 — Vérifier un mot de passe

```
<?php
$mdp = "biblio1234";
if (strlen($mdp) < 8) {
    echo "Refusé : moins de 8 caractères";
} elseif (strpos($mdp, "1234") !== false) {
    echo "Refusé : contient 1234";
} else {
    echo "Accepté";
}
?>
```

strpos peut retourner 0 (motif au début) : on compare donc avec !== false et non avec == false. « biblio1234 » est refusé, « Carthage2026 » est accepté.

Chapitre P6 — Les tableaux indicés et associatifs

OBJECTIFS DU CHAPITRE

- Créer un tableau indicé et un tableau associatif.
- Lire, modifier et ajouter un élément.
- Compter les éléments avec count.
- Parcourir un tableau et l'afficher dans un tableau HTML.

P6.1 Le tableau indicé

Les éléments sont repérés par un indice numérique qui commence à 0.

```
$notes = array(12, 15.5, 9, 14);   // avec array()
$jours = ["Lundi", "Mardi"];       // écriture courte équivalente
echo $notes[0];                    // 12
$notes[2] = 10;                    // modification
$notes[] = 17;                     // ajout à la fin
echo count($notes);                // 5
```

P6.2 Le tableau associatif

Les éléments sont repérés par une **clé** (une chaîne) au lieu d'un indice. On associe clé et valeur avec `=>`.

```
$livre = [
    "titre" => "Le Petit Prince",
    "auteur" => "Antoine de Saint-Exupéry",
    "annee" => 1943
];
echo $livre["titre"];           // Le Petit Prince
$livre["genre"] = "Roman";      // ajout d'une clé
```

À RETENIR

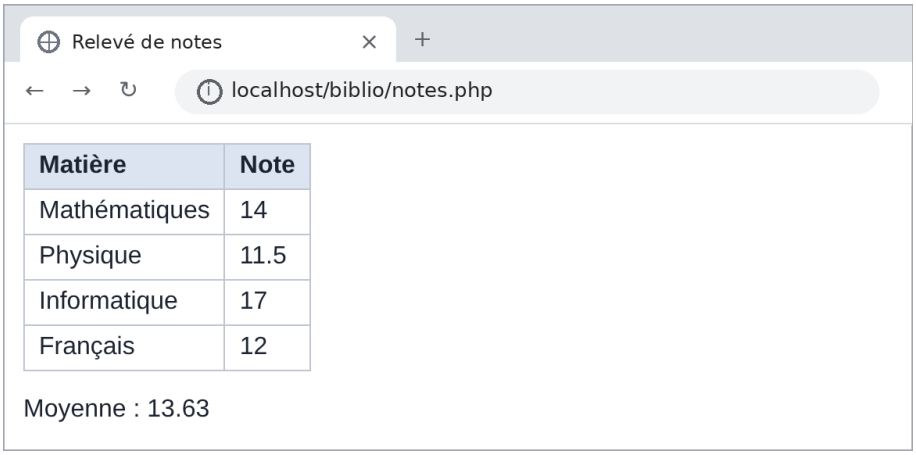
$_GET, $_POST et chaque ligne lue dans une base de données sont des tableaux associatifs : cette notion est la clé des parties 2 et 3.

P6.3 Parcourir un tableau

Un tableau indicé se parcourt avec for, de 0 à count($t) - 1. Un tableau associatif se parcourt avec `foreach`, qui donne à chaque tour la clé et la valeur.

notes.php

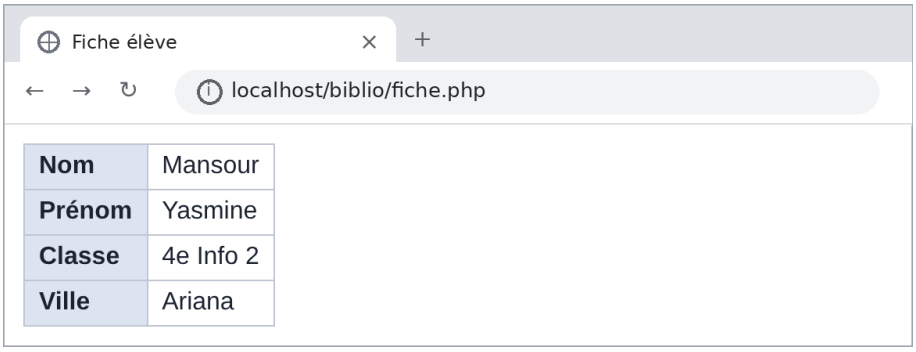
```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Relevé de notes</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
<?php
$matieres = ["Mathématiques", "Physique", "Informatique", "Français"];
$notes = [14, 11.5, 17, 12];
$somme = 0;
echo "<table><tr><th>Matière</th><th>Note</th></tr>";
for ($i = 0; $i < count($notes); $i++) {
    echo "<tr><td>" . $matieres[$i] . "</td><td>" . $notes[$i] . "</td></tr>";
    $somme = $somme + $notes[$i];
}
echo "</table>";
echo "<p>Moyenne : " . round($somme / count($notes), 2) . "</p>";
?>
</body>
</html>
```



Résultat dans le navigateur

fiche.php

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Fiche élève</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
<?php
$eleve = ["Nom" => "Mansour", "Prénom" => "Yasmine", "Classe" => "4e Info 2",
"Ville" => "Ariana"];
echo "<table>";
foreach ($eleve as $cle => $valeur) {
    echo "<tr><th>" . $cle . "</th><td>" . $valeur . "</td></tr>";
}
echo "</table>";
?>
</body>
</html>
```



Résultat dans le navigateur

ATTENTION

foreach complète les boucles du programme (for, while, do ... while) : c'est la façon naturelle de parcourir un tableau associatif.

Synthèse

À RETENIR

- Indicé : array(...) ou [ ], indices de 0 à count - 1, ajout avec $t[] = v.
- Associatif : "clé" => valeur, accès $t["clé"].
- Parcours : for (indicé), foreach ($t as $cle => $valeur) (associatif).

Quiz

1. $t = [5, 8, 2]; que vaut $t[1] ?

a) 5
b) 8
c) 2
d) Une erreur

2. Quelle fonction donne le nombre d'éléments ?

a) strlen
b) count
c) length
d) size

3. Comment lire la valeur de la clé "ville" ?

a) $t.ville
b) $t["ville"]
c) $t(ville)
d) $t->ville

Exercices

Exercice 1 — Le relevé de notes

Rangez les notes 11, 13.5, 8, 17 et 15 dans un tableau. Affichez-les dans un tableau HTML (une ligne par note), puis la moyenne arrondie à 2 décimales et la meilleure note.

Exercice 2 — La fiche d'un élève

Créez un tableau associatif décrivant un livre (titre, auteur, année, genre). Ajoutez la clé « disponible » avec la valeur « oui ». Affichez la fiche dans un tableau HTML avec foreach, puis le nombre d'informations avec count.

Corrigés

Quiz

1. b) Indices à partir de 0.

2. b) count($t).

3. b) Crochets et clé entre guillemets.

Exercice 1 — Le relevé de notes

```
<?php
$notes = [11, 13.5, 8, 17, 15];
$somme = 0;
$max = $notes[0];
echo "<table>";
for ($i = 0; $i < count($notes); $i++) {
    echo "<tr><td>Note " . ($i + 1) . "</td><td>" . $notes[$i] . "</td></tr>";
    $somme = $somme + $notes[$i];
    if ($notes[$i] > $max) { $max = $notes[$i]; }
}
echo "</table>";
echo "<p>Moyenne : " . round($somme / count($notes), 2) . " · Meilleure note :
" . $max . "</p>";
?>
```

Moyenne : 12.9 ; meilleure note : 17.

Exercice 2 — La fiche d'un élève

```
<?php
$livre = ["Titre" => "Le Prophète", "Auteur" => "Khalil Gibran", "Année" => 1923,
"Genre" => "Poésie"];
$livre["Disponible"] = "oui";
echo "<table>";
foreach ($livre as $cle => $valeur) {
    echo "<tr><th>$cle</th><td>$valeur</td></tr>";
}
echo "</table><p>" . count($livre) . " informations</p>";
?>
```

Chapitre P7 — Les structures de contrôle

OBJECTIFS DU CHAPITRE

- Écrire une condition avec les opérateurs de comparaison et logiques.
- Utiliser if, elseif, else.
- Répéter avec for, while et do ... while.
- Utiliser les fonctions mathématiques abs, sqrt, round et rand.

P7.1 Opérateurs de comparaison et logiques

Comparaison	Signification	Logique	Signification
==	égal à	&&	ET
<> ou !=	différent de	||	OU
< <=	inférieur, inférieur ou égal	!	NON
> >=	supérieur, supérieur ou égal		

ATTENTION

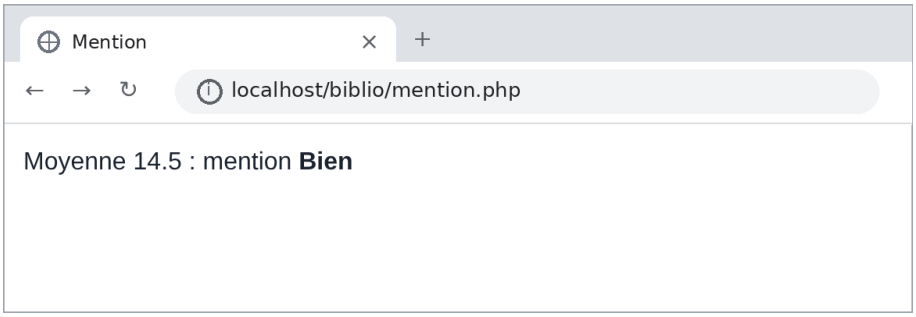
= affecte, == compare. === (identique) compare aussi le type : il est utile avec strpos, qui peut retourner 0 ou false.

P7.2 La structure if

mention.php

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Mention</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
<?php
$moyenne = 14.5;
if ($moyenne >= 16) {
    $mention = "Très bien";
} elseif ($moyenne >= 14) {
    $mention = "Bien";
} elseif ($moyenne >= 12) {
    $mention = "Assez bien";
} elseif ($moyenne >= 10) {
    $mention = "Passable";
} else {
    $mention = "Non admis";
```

```
}
echo "<p>Moyenne $moyenne : mention <b>$mention</b></p>";
?>
</body>
</html>
```



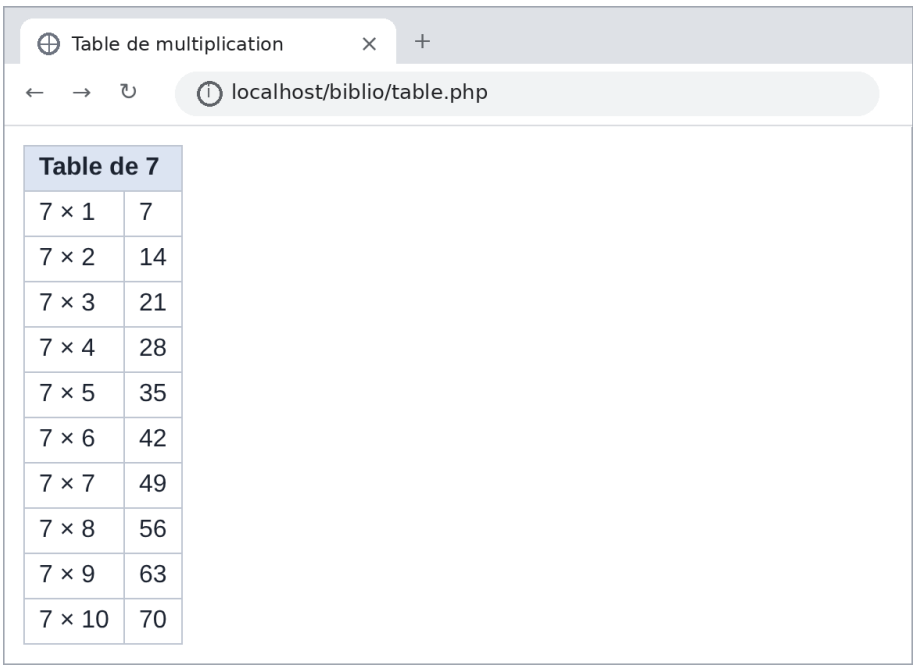
Résultat dans le navigateur

P7.3 Les boucles for, while et do ... while

Boucle	Quand l'utiliser
for ($i = 1; $i <= 10; $i++) { ... }	Nombre de répétitions connu
while (condition) { ... }	Tant qu'une condition est vraie (testée avant : 0 passage possible)
do { ... } while (condition);	Au moins un passage, puis tant que la condition est vraie

table.php

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Table de multiplication</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
<?php
$n = 7;
echo "<table><tr><th colspan='2'>Table de $n</th></tr>";
for ($i = 1; $i <= 10; $i++) {
    echo "<tr><td>$n × $i</td><td>" . ($n * $i) . "</td></tr>";
}
echo "</table>";
?>
</body>
</html>
```

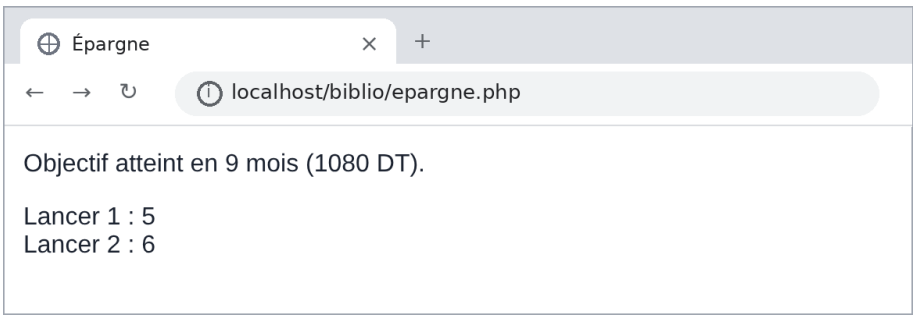


Résultat dans le navigateur

epargne.php

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Épargne</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
<?php
// while : combien de mois pour atteindre l'objectif ?
$objectif = 1000;
$solde = 0;
$mois = 0;
while ($solde < $objectif) {
    $solde = $solde + 120;
    $mois++;
}
echo "<p>Objectif atteint en $mois mois ($solde DT).</p>";

// do … while : lancer un dé jusqu'à obtenir 6
$lancers = 0;
do {
    $de = rand(1, 6);
    $lancers++;
    echo "Lancer $lancers : $de<br>";
} while ($de <> 6);
?>
</body>
</html>
```



Un tirage : les lancers changent à chaque rechargement

P7.4 Les fonctions mathématiques

Fonction	Rôle	Exemple	Résultat
abs(x)	Valeur absolue	abs(-7)	7
sqrt(x)	Racine carrée	sqrt(49)	7
round(x, n)	Arrondi (n décimales, facultatif)	round(12.456, 2)	12.46
rand(min, max)	Entier aléatoire entre min et max	rand(1, 6)	1 à 6

Synthèse

À RETENIR

- == <> != < <= > >= · && || !
- if / elseif / else ; for (nombre connu), while (tant que), do … while (au moins une fois).
- abs, sqrt, round, rand.

Quiz

1. Combien de passages pour for ($i = 0; $i < 5; $i++) ?

a) 4
b) 5
c) 6
d) Infini

2. Quelle boucle s'exécute au moins une fois ?

a) for
b) while
c) do … while
d) Aucune

3. Que retourne rand(1, 3) ?

a) Un réel entre 1 et 3

b) 1, 2 ou 3

c) Toujours 2

d) 0 à 3

Exercices

Exercice 1 — La mention au bac

La variable $moyenne contient une moyenne. Affichez « Admis » si elle est supérieure ou égale à 10, « Session de contrôle » si elle est entre 7 (inclus) et 10, « Refusé » sinon. Testez avec 15, 8.5 et 6.

Exercice 2 — La table de multiplication

Affichez dans un tableau HTML les tables de 1 à 5 : une ligne par multiplicateur (de 1 à 10), une colonne par table. Utilisez deux boucles for imbriquées.

Exercice 3 — Le lancer de dé

Lancez deux dés avec rand jusqu'à obtenir un double (deux valeurs égales). Affichez chaque lancer, puis le nombre de lancers nécessaires.

Corrigés

Quiz

1. b) $i vaut 0 à 4.

2. c) Condition testée après le bloc.

3. b) Entier aléatoire, bornes comprises.

Exercice 1 — La mention au bac

```
<?php
$moyenne = 8.5;
if ($moyenne >= 10) {
    echo "Admis";
} elseif ($moyenne >= 7) {
    echo "Session de contrôle";
} else {
    echo "Refusé";
}
?>
```

Exercice 2 — La table de multiplication

```
<?php
echo "<table><tr><th></th>";
for ($t = 1; $t <= 5; $t++) { echo "<th>× $t</th>"; }
echo "</tr>";
for ($i = 1; $i <= 10; $i++) {
    echo "<tr><th>$i</th>";
    for ($t = 1; $t <= 5; $t++) { echo "<td>" . ($i * $t) . "</td>"; }
    echo "</tr>";
}
echo "</table>";
?>
```

Exercice 3 — Le lancer de dé

```
<?php
$n = 0;
do {
    $d1 = rand(1, 6);
    $d2 = rand(1, 6);
    $n++;
    echo "Lancer $n : $d1 et $d2<br>";
} while ($d1 <> $d2);
echo "Double obtenu en $n lancers.";
?>
```

do … while : il faut lancer au moins une fois avant de pouvoir tester.

Chapitre P8 — La date et l'heure

OBJECTIFS DU CHAPITRE

- Afficher la date et l'heure avec date.
- Obtenir un timestamp avec time et strtotime.
- Calculer un écart entre deux dates.
- Vérifier la validité d'une date avec checkdate.

P8.1 Le fuseau horaire

Pour obtenir l'heure de Tunis, on fixe le fuseau horaire en tête du script avec `date_default_timezone_set("Africa/Tunis");`. Cette instruction ne figure pas dans l'annexe, mais sans elle l'heure affichée peut être décalée.

P8.2 Formater une date avec date()

Caractère	Signification	Exemple
d	Jour du mois sur 2 chiffres	07
m	Mois sur 2 chiffres	10
Y	Année sur 4 chiffres	2026
H	Heure (00 à 23)	14
i	Minutes	05
s	Secondes	42
N	Jour de la semaine (1 = lundi … 7 = dimanche)	3

P8.3 Timestamp, écart entre dates et validité

- `time()` : l'instant présent en timestamp (nombre de secondes écoulées depuis le 1er janvier 1970 à 00:00:00).
- `strtotime("2026-12-20")` : le timestamp d'une date écrite en toutes lettres. La différence de deux timestamps, divisée par 86400 (secondes dans un jour), donne un nombre de jours.
- `date("d/m/Y", $timestamp)` : formate un timestamp donné.
- `checkdate($mois, $jour, $annee)` : true si la date existe. Attention à l'ordre : mois, jour, année.

date.php

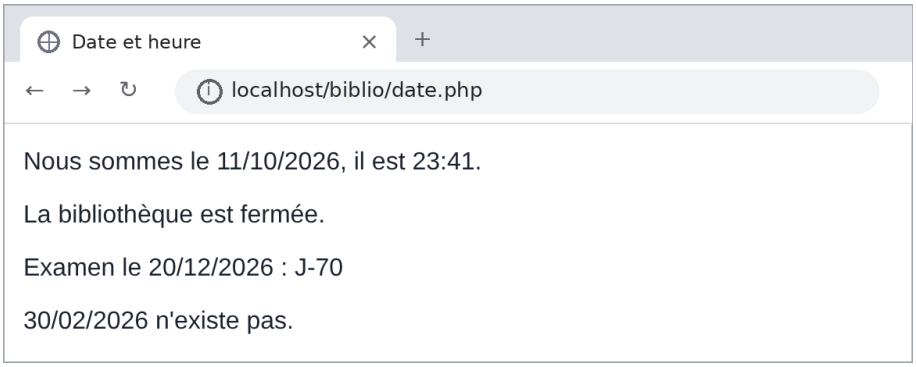
```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
```

```
  <title>Date et heure</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
<?php
date_default_timezone_set("Africa/Tunis");
echo "<p>Nous sommes le " . date("d/m/Y") . ", il est " . date("H:i") . ".</p>";

// Ouvert ou fermé ? (du lundi au samedi, de 8h à 18h)
$heure = (int) date("H");
$jour = (int) date("N");
if ($jour <= 6 && $heure >= 8 && $heure < 18) {
    echo "<p class='ok'>La bibliothèque est ouverte.</p>";
} else {
    echo "<p>La bibliothèque est fermée.</p>";
}

// Jours avant l'examen
$examen = strtotime("2026-12-20");
$jours = ceil(($examen - time()) / 86400);
echo "<p>Examen le " . date("d/m/Y", $examen) . " : J-$jours</p>";

// Validité d'une date
if (checkdate(2, 30, 2026)) { echo "<p>30/02/2026 existe.</p>"; }
else { echo "<p>30/02/2026 n'existe pas.</p>"; }
?>
</body>
</html>
```



Le résultat dépend du jour et de l'heure de l'exécution

ceil arrondit à l'entier supérieur : un examen dans 2 jours et 3 heures compte pour J-3. Si vous préférez rester dans l'annexe, round donne un résultat proche.

Synthèse

À RETENIR

- date_default_timezone_set("Africa/Tunis") en tête du script.
- date("d/m/Y H:i") ; time() et strtotime(…) donnent des timestamps.
- Écart en jours : (t2 - t1) / 86400 ; checkdate(mois, jour, année).

Quiz

1. Que retourne date("Y") en 2026 ?

a) 26
b) 2026
c) "Y"
d) Un timestamp

2. Qu'est-ce qu'un timestamp ?

a) Une date en lettres
b) Un nombre de secondes depuis le 1er janvier 1970
c) Un fuseau horaire
d) Une heure arrondie

3. checkdate(13, 1, 2026) retourne :

a) true
b) false
c) 13
d) Une erreur

Exercices

Exercice 1 — La date du jour

Affichez en haut de page « Aujourd'hui : jj/mm/aaaa, hh:mm ». Puis affichez l'âge, cette année, d'une personne née en 2008.

Exercice 2 — Ouvert ou fermé ?

Un cybercafé ouvre du lundi au samedi de 9h à 22h, et le dimanche de 14h à 20h. Affichez « Ouvert » ou « Fermé » selon le jour et l'heure actuels.

Exercice 3 — Avant l'examen

Demandez (dans le code) une date d'examen sous la forme jour, mois, année. Vérifiez d'abord qu'elle existe avec checkdate ; si oui, affichez le nombre de jours restants, ou « Examen passé » si la date est dépassée.

Corrigés

Quiz

1. b) Y : année sur 4 chiffres.

2. b) Voir l'annexe.

3. b) Le mois 13 n'existe pas.

Exercice 1 — La date du jour

```php
<?php
date_default_timezone_set("Africa/Tunis");
echo "Aujourd'hui : " . date("d/m/Y, H:i") . "<br>";
echo "Âge en " . date("Y") . " : " . ((int) date("Y") - 2008) . " ans";
?>
```

Exercice 2 — Ouvert ou fermé ?

```php
<?php
date_default_timezone_set("Africa/Tunis");
$h = (int) date("H");
$j = (int) date("N");
if (($j <= 6 && $h >= 9 && $h < 22) || ($j == 7 && $h >= 14 && $h < 20)) {
    echo "Ouvert";
} else {
    echo "Fermé";
}
?>
```

Exercice 3 — Avant l'examen

```php
<?php
date_default_timezone_set("Africa/Tunis");
$j = 20; $m = 12; $a = 2026;
if (!checkdate($m, $j, $a)) {
    echo "Date invalide";
} else {
    $t = strtotime("$a-$m-$j");
    $reste = round(($t - time()) / 86400);
    if ($reste < 0) { echo "Examen passé"; } else { echo "J-$reste"; }
}
?>
```

Chapitre P9 — Importer un fichier avec require

OBJECTIFS DU CHAPITRE

- Expliquer l'intérêt de découper un site en fichiers réutilisables.
- Importer un fichier avec require.
- Partager un en-tête, un menu et un pied de page entre plusieurs pages.

P9.1 Le principe

L'instruction `require "fichier.php";` insère et exécute le contenu d'un autre fichier à cet endroit. Un en-tête modifié une seule fois change sur toutes les pages. Si le fichier est introuvable, require arrête le script avec une erreur.

P9.2 Exemple : un en-tête commun

L'en-tête utilise la variable $titre, définie dans chaque page avant l'import : le titre de l'onglet change d'une page à l'autre.

entete.php

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title><?php echo $titre; ?></title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
```

menu.php

```
<nav>
  <a href="index.php">Accueil</a> |
  <a href="catalogue.php">Catalogue</a> |
  <a href="contact.php">Contact</a>
</nav>
```

pied.php

```
</body>
</html>
```

index.php

```php
<?php
$titre = "Accueil";
require "entete.php";
require "menu.php";
?>
<h1>Bibliothèque municipale</h1>
<p>Bienvenue ! Consultez notre catalogue en ligne.</p>
<?php require "pied.php"; ?>
```



index.php assemblée à partir de quatre fichiers

À RETENIR

Dans la partie 3, le fichier connexion.php, importé avec require, contiendra la connexion à la base : il sera écrit une seule fois et utilisé par toutes les pages.

Synthèse

À RETENIR

- require "fichier.php"; insère et exécute un fichier.
- En-tête, menu, pied de page et connexion : écrits une fois, importés partout.
- Fichier absent : erreur et arrêt du script.

Quiz

1. Que fait require "menu.php"; ?

a) Ouvre menu.php dans un nouvel onglet

b) Insère et exécute menu.php à cet endroit

c) Crée menu.php

d) Supprime menu.php

2. Si le fichier importé n'existe pas ?

a) Rien ne se passe

b) Erreur et arrêt du script

c) Il est créé

d) La page est vide sans erreur

Exercices

Exercice 1 — Un site de trois pages

Créez entete.php (avec la variable $titre dans <title>), menu.php et pied.php (« © Bibliothèque municipale » suivi de l'année courante avec date). Créez index.php, catalogue.php et contact.php qui importent ces trois fichiers et n'ont chacune que leur propre contenu.

Corrigés

Quiz

1. b) Le contenu est importé.

2. b) require est bloquant.

Exercice 1 — Un site de trois pages

```
<?php
// catalogue.php
$titre = "Catalogue";
require "entete.php";
require "menu.php";
?>
<h1>Catalogue</h1>
<p>La liste des livres sera lue dans la base (partie 3).</p>
<?php require "pied.php"; ?>
```

```
<!-- pied.php -->
<footer>© Bibliothèque municipale <?php echo date("Y"); ?></footer>
</body>
</html>
```

Partie 2 — Transmettre des données entre pages

Une page PHP peut recevoir des données d'une autre page : par l'adresse (URL) ou par un formulaire. Ces deux mécanismes préparent la partie 3, où les données reçues serviront à interroger et à modifier une base de données.

Chapitre P10 — Le principe de la transmission

OBJECTIFS DU CHAPITRE

- Distinguer les méthodes GET et POST.
- Lire une URL avec paramètres.
- Définir les variables superglobales $_GET et $_POST.
- Vérifier qu'une donnée a été transmise avec isset.

P10.1 Deux méthodes HTTP

	GET	POST
Où sont les données	Dans l'URL, après le ?	Dans le corps de la requête, invisibles dans l'URL
Exemple	livre.php?id=3&format=court	Formulaire avec method="post"
Usage	Lien vers une fiche, recherche, pagination	Saisie d'un formulaire, ajout ou modification de données
Lues dans	$_GET	$_POST

P10.2 Les paramètres d'URL

Après le nom de la page, `?` introduit les paramètres, écrits `nom=valeur` et séparés par `&`.

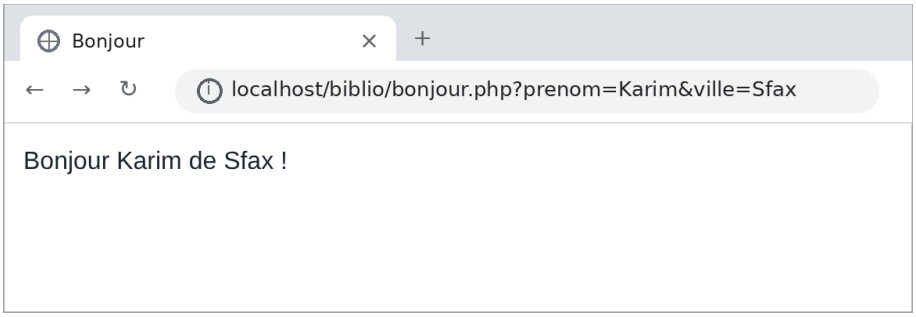
```
http://localhost/biblio/bonjour.php?prenom=Karim&ville=Sfax
                                   └──────┬──────┘ └──┬──┘
                                  paramètre 1   paramètre 2
```

P10.3 Les variables superglobales

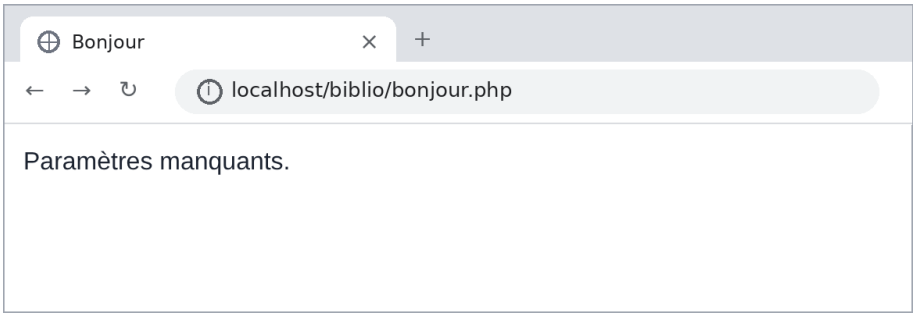
$_GET et $_POST sont des **tableaux associatifs** remplis automatiquement par PHP et disponibles dans toutes les pages. La clé est le nom du paramètre (ou l'attribut name du champ), la valeur est la donnée transmise, toujours sous forme de chaîne.

bonjour.php

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Bonjour</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
<?php
if (isset($_GET["prenom"]) && isset($_GET["ville"])) {
    echo "<p>Bonjour " . $_GET["prenom"] . " de " . $_GET["ville"] . " !</p>";
} else {
    echo "<p>Paramètres manquants.</p>";
}
?>
</body>
</html>
```



Avec les deux paramètres dans l'URL



Sans paramètre : isset évite l'erreur

ATTENTION

Lire $_GET["prenom"] alors que le paramètre est absent provoque l'avertissement « Undefined array key ». Testez toujours avec isset avant de lire une donnée transmise. Pour arrêter une page à laquelle il manque une donnée indispensable : die("Message");

Synthèse

À RETENIR

- GET : données dans l'URL (?nom=valeur&…) ; POST : données invisibles, envoyées par un formulaire.
- $_GET et $_POST : tableaux associatifs superglobaux ; valeurs toujours des chaînes.
- isset avant de lire ; die pour arrêter la page.

Quiz

1. Dans page.php?ville=Gabes, comment lire la ville ?

a) $_POST["ville"]
b) $_GET["ville"]
c) $ville
d) $_GET->ville

2. Quel caractère sépare deux paramètres d'URL ?

a) ?
b) &
c) ;
d) ,

3. Quelle méthode n'affiche pas les données dans l'URL ?

a) GET
b) POST
c) Les deux
d) Aucune

Exercices

Exercice 1 — Lire une URL

Pour l'URL http://localhost/boutique/produit.php?id=4&categorie=Stockage&tri=prix : a) quelle page est demandée ? b) combien de paramètres sont transmis ? c) écrivez l'instruction qui affiche la catégorie ; d) écrivez le test qui vérifie que l'identifiant a été transmis.

Corrigés

Quiz

1. b) Paramètre d'URL : $_GET.

2. b) ? les introduit, & les sépare.

3. b) POST les envoie dans le corps de la requête.

Exercice 1 — Lire une URL

a) produit.php ; b) trois (id, categorie, tri) ; c) echo $_GET["categorie"]; ; d) if (isset($_GET["id"])) { … }

Chapitre P11 — Transmettre par l'URL avec $_GET

OBJECTIFS DU CHAPITRE

- Construire des liens avec paramètres.
- Lire un paramètre sur la page cible.
- Gérer un paramètre absent ou incorrect.

P11.1 Exemple : du catalogue à la fiche

La page catalogue.php génère un lien par livre ; chaque lien transmet l'indice du livre à livre.php. Ici, les livres sont rangés dans un tableau ; dans la partie 3, ils seront lus dans une base de données.

livres.php

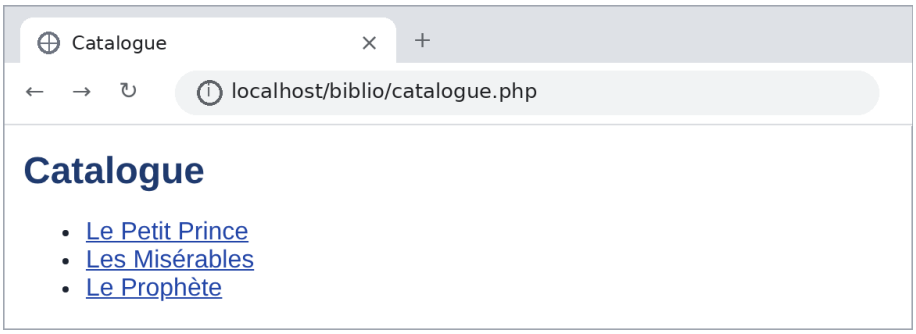
```
<?php
// livres.php : données de démonstration (remplacées par la base en partie 3)
$livres = [
    ["titre" => "Le Petit Prince", "auteur" => "Antoine de Saint-Exupéry",
"annee" => 1943],
    ["titre" => "Les Misérables", "auteur" => "Victor Hugo", "annee" => 1862],
    ["titre" => "Le Prophète", "auteur" => "Khalil Gibran", "annee" => 1923]
];
?>
```

catalogue.php

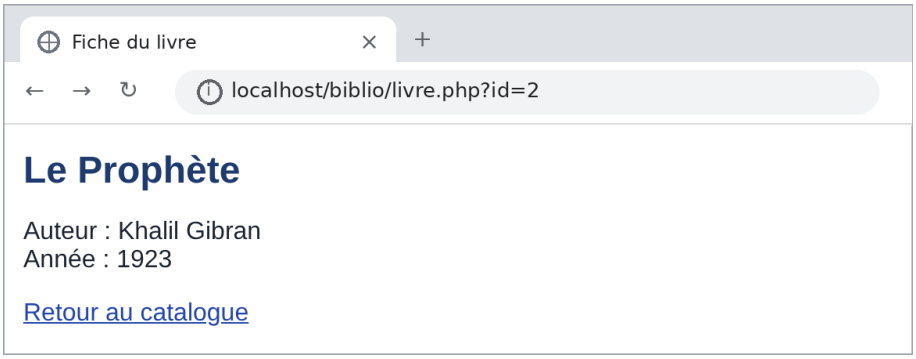
```
<?php
$titre = "Catalogue";
require "entete.php";
require "livres.php";
echo "<h1>Catalogue</h1><ul>";
for ($i = 0; $i < count($livres); $i++) {
    echo '<li><a href="livre.php?id=' . $i . '">' . $livres[$i]["titre"] .
'</a></li>';
}
echo "</ul>";
require "pied.php";
?>
```

livre.php

```
<?php
$titre = "Fiche du livre";
require "entete.php";
require "livres.php";
if (!isset($_GET["id"])) {
    die("<p class='erreur'>Aucun livre choisi.</p>");
}
$id = (int) $_GET["id"];
if ($id < 0 || $id >= count($livres)) {
    die("<p class='erreur'>Livre introuvable.</p>");
}
$livre = $livres[$id];
echo "<h1>" . $livre["titre"] . "</h1>";
echo "<p>Auteur : " . $livre["auteur"] . "<br>Année : " . $livre["annee"] .
"</p>";
echo '<p><a href="catalogue.php">Retour au catalogue</a></p>';
require "pied.php";
?>
```



catalogue.php : chaque lien transmet un id



Après un clic sur « Le Prophète » : livre.php?id=2

À RETENIR

(int) convertit le paramètre en entier : un id modifié à la main dans l'URL (« abc ») devient 0, puis le test de l'intervalle refuse les valeurs impossibles. Ce réflexe protégera aussi les requêtes SQL de la partie 3.

Synthèse

À RETENIR

- Lien : <a href="page.php?id=3">.
- Page cible : isset($_GET["id"]), puis (int) $_GET["id"].
- Paramètre absent ou impossible : message et die.

Quiz

1. Quel lien transmet id = 5 à fiche.php ?

a) <a href="fiche.php&id=5">
b) <a href="fiche.php?id=5">
c) <a href="fiche.php/5">
d) <a href="fiche.php" id="5">

2. Que vaut (int) $_GET["id"] pour fiche.php?id=abc ?

a) "abc"
b) 0
c) false
d) Une erreur

Exercices

Exercice 1 — La fiche produit

Créez produits.php contenant un tableau de quatre produits (nom, prix, stock). La page liste.php affiche un lien par produit vers produit.php?id=..., qui affiche la fiche complète et « En stock » ou « Rupture de stock » selon le stock. Gérez un id absent ou invalide.

Exercice 2 — Le menu du jour

La page menu.php affiche cinq liens : Lundi … Vendredi, vers menu.php?jour=1 … ?jour=5. Quand un jour est transmis, la page affiche aussi le plat correspondant (tableau indicé de 5 plats). Sans paramètre, elle affiche « Choisissez un jour ».

Corrigés

Quiz

1. b) ? puis nom=valeur.

2. b) Une chaîne non numérique donne 0.

Exercice 1 — La fiche produit

```
<?php
// produit.php
require "produits.php";
if (!isset($_GET["id"])) { die("Aucun produit choisi."); }
$id = (int) $_GET["id"];
if ($id < 0 || $id >= count($produits)) { die("Produit introuvable."); }
$p = $produits[$id];
echo "<h1>" . $p["nom"] . "</h1><p>Prix : " . $p["prix"] . " DT</p>";
if ($p["stock"] > 0) { echo "<p>En stock (" . $p["stock"] . ")</p>"; }
else { echo "<p>Rupture de stock</p>"; }
?>
```

Exercice 2 — Le menu du jour

```
<?php
$jours = ["Lundi", "Mardi", "Mercredi", "Jeudi", "Vendredi"];
$plats = ["Couscous", "Lablabi", "Pâtes", "Riz au poisson", "Ojja"];
for ($i = 0; $i < 5; $i++) {
    echo '<a href="menu.php?jour=' . ($i + 1) . '">' . $jours[$i] . '</a> ';
}
if (isset($_GET["jour"])) {
    $j = (int) $_GET["jour"];
    if ($j >= 1 && $j <= 5) { echo "<p>" . $jours[$j - 1] . " : " . $plats[$j -
1] . "</p>"; }
} else {
    echo "<p>Choisissez un jour</p>";
}
?>
```

Chapitre P12 — Transmettre par un formulaire avec $_POST

OBJECTIFS DU CHAPITRE

- Relier un formulaire à une page de traitement avec action et method.
- Lire les champs dans $_POST grâce à leur attribut name.
- Traiter le formulaire sur la même page.
- Contrôler les données reçues avant de les utiliser.

P12.1 Le formulaire et sa page de traitement

- `action="traitement.php"` : la page qui reçoit les données (vide ou absente : la page elle-même).
- `method="post"` : les données passent par $_POST.
- `name="nom"` : la clé de la donnée dans $_POST. Un champ sans name n'est pas transmis.

P12.2 Exemple : l'inscription à la bibliothèque

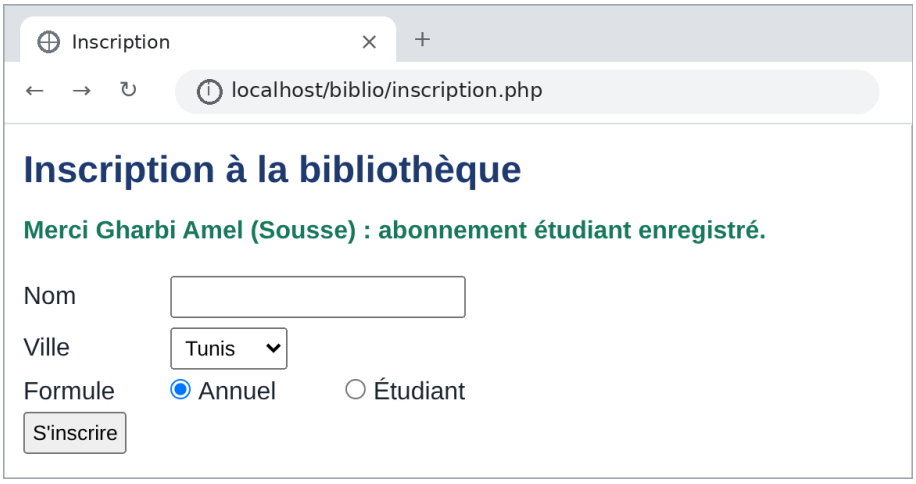
Le formulaire et son traitement sont dans la même page. Le bouton porte un name : isset($_POST["envoyer"]) indique que le formulaire vient d'être envoyé.

inscription.php

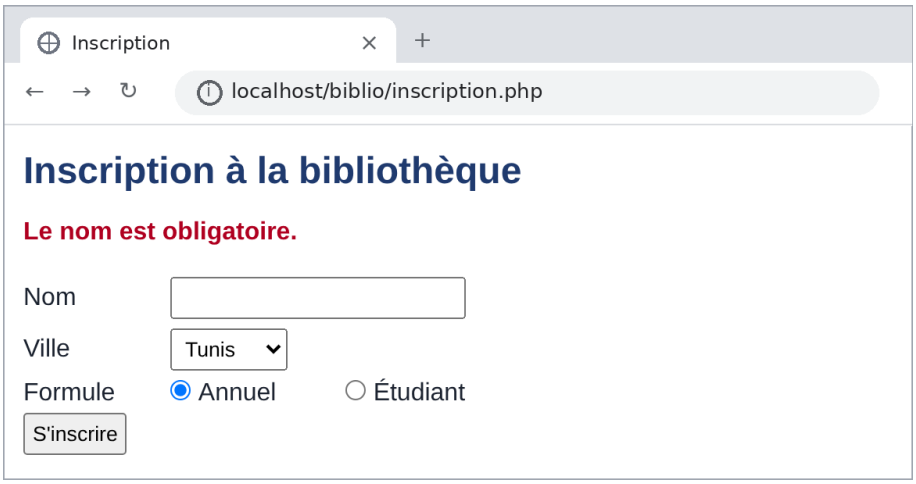
```
<?php
$titre = "Inscription";
require "entete.php";
echo "<h1>Inscription à la bibliothèque</h1>";

if (isset($_POST["envoyer"])) {
    $nom = trim($_POST["nom"]);
    $ville = $_POST["ville"];
    $formule = $_POST["formule"];
    if ($nom == "") {
        echo "<p class='erreur'>Le nom est obligatoire.</p>";
    } else {
        echo "<p class='ok'>Merci $nom ($ville) : abonnement $formule
enregistré.</p>";
    }
}
?>
<form method="post" action="">
  <label for="nom">Nom</label>
  <input type="text" name="nom" id="nom"><br>
  <label for="ville">Ville</label>
  <select name="ville" id="ville">
    <option>Tunis</option><option>Sfax</option><option>Sousse</option>
  </select><br>
  <label>Formule</label>
  <input type="radio" name="formule" value="annuel" id="f1" checked> <label
for="f1">Annuel</label>
  <input type="radio" name="formule" value="étudiant" id="f2"> <label
for="f2">Étudiant</label><br>
  <input type="submit" name="envoyer" value="S'inscrire">
```

```
</form>
<?php require "pied.php"; ?>
```



Après l'envoi : les données sont lues dans $_POST



Nom laissé vide : le contrôle refuse la saisie

ATTENTION

Les données d'un formulaire viennent de l'utilisateur : contrôlez-les toujours (champ vide avec trim, nombre avec (int) ou (float), valeur attendue) avant de les afficher ou de les enregistrer dans une base.

P12.3 GET ou POST ?

Situation	Méthode
Lien vers la fiche d'un livre, recherche que l'on veut pouvoir partager	GET
Formulaire d'inscription, de contact, mot de passe	POST
Ajout, modification ou suppression de données dans une base	POST (ou GET avec confirmation, chapitre P18)

Synthèse

À RETENIR

- <form method="post" action="..."> ; chaque champ a un name.
- isset($_POST["envoyer"]) : le formulaire vient d'être envoyé.
- Contrôler avant d'utiliser : trim, test de vide, transtypage.

Quiz

1. Quel attribut donne la clé d'un champ dans $_POST ?

a) id
b) name
c) value
d) for

2. action="" signifie :

a) Le formulaire n'est pas envoyé
b) Les données sont envoyées à la page elle-même
c) Erreur
d) Envoi par GET

3. Pour savoir si le formulaire vient d'être envoyé :

a) isset($_POST["envoyer"])
b) $_POST == true
c) isset($envoyer)
d) echo $_POST

Exercices

Exercice 1 — Le formulaire de contact

Créez contact.php : nom, e-mail, objet (liste : Renseignement, Réservation, Réclamation) et message. Après l'envoi, si le nom ou le message est vide, affichez une erreur ; sinon, affichez un récapitulatif dans un tableau HTML.

Exercice 2 — La conversion de monnaie

Créez un formulaire (méthode POST) avec un montant en dinars et une liste de devises (Euro, Dollar). Taux d'exemple : 1 € = 3.4 DT, 1 $ = 3.1 DT. Après l'envoi, affichez le montant converti arrondi à 2 décimales, ou une erreur si le montant n'est pas un nombre positif.

Corrigés

Quiz

1. b) La clé est l'attribut name.

2. b) Traitement sur la même page.

3. a) Le bouton nommé envoyer n'est présent qu'après l'envoi.

Exercice 1 — Le formulaire de contact

```
<?php
if (isset($_POST["envoyer"])) {
    $nom = trim($_POST["nom"]);
    $message = trim($_POST["message"]);
    if ($nom == "" || $message == "") {
        echo "<p class='erreur'>Nom et message obligatoires.</p>";
    } else {
        echo "<table>";
        echo "<tr><th>Nom</th><td>$nom</td></tr>";
        echo "<tr><th>E-mail</th><td>" . $_POST["email"] . "</td></tr>";
        echo "<tr><th>Objet</th><td>" . $_POST["objet"] . "</td></tr>";
        echo "<tr><th>Message</th><td>$message</td></tr>";
        echo "</table>";
    }
}
?>
```

Exercice 2 — La conversion de monnaie

```
<?php
if (isset($_POST["convertir"])) {
    $montant = (float) $_POST["montant"];
    if ($montant <= 0) {
        echo "<p class='erreur'>Montant invalide.</p>";
    } else {
        if ($_POST["devise"] == "Euro") { $res = $montant / 3.4; $sym = "€"; }
        else { $res = $montant / 3.1; $sym = "$"; }
        echo "<p>$montant DT = " . round($res, 2) . " $sym</p>";
    }
}
?>
```

Partie 3 — Interagir avec une base de données

C'est la partie principale du cours. Une application web lit et enregistre ses données dans une base MySQL : le catalogue d'une bibliothèque, les produits d'une boutique, les inscrits d'un centre de formation. Chaque chapitre associe une requête SQL à un formulaire : rechercher, ajouter, supprimer, modifier.

Chapitre	Requête	Formulaire ou lien
P13 — La base dans phpMyAdmin	Toutes, en mode assisté	Onglets de phpMyAdmin
P14 — Se connecter	—	—
P15 — Afficher des données	SELECT	Lien vers une fiche ($_GET)
P16 — Rechercher	SELECT … WHERE	Formulaire de recherche ($_GET)
P17 — Ajouter	INSERT	Formulaire d'ajout ($_POST)
P18 — Supprimer	DELETE	Lien + formulaire de confirmation
P19 — Modifier	UPDATE	Formulaire pré-rempli ($_POST)
P20 — Application complète	Les quatre	Une page de gestion

Chapitre P13 — La base de données dans phpMyAdmin

OBJECTIFS DU CHAPITRE

- Rappeler les notions de base, table, champ, type et clé primaire.
- Créer une base et ses tables avec phpMyAdmin.
- Générer une requête SQL en mode assisté et la réutiliser dans PHP.

P13.1 Rappels

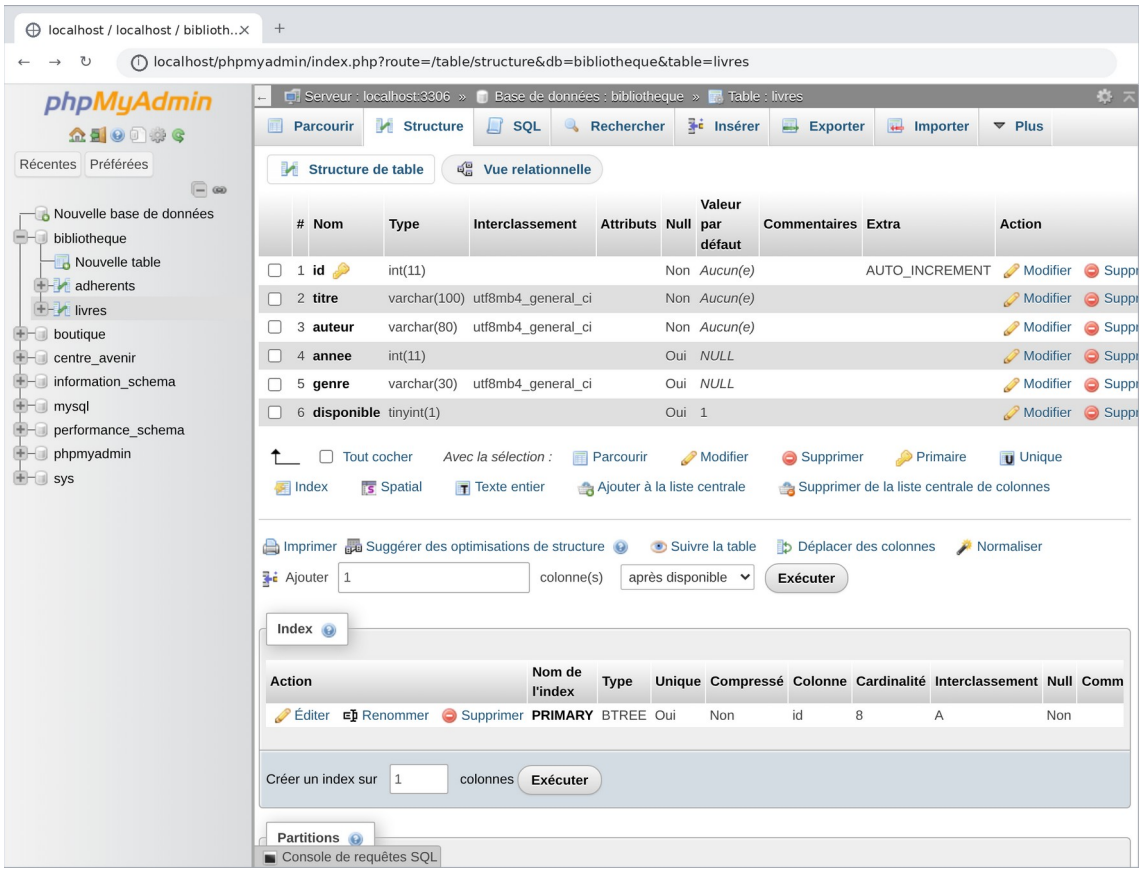
Notion	Définition	Exemple
Base de données	Ensemble de tables gérées par le SGBD	bibliotheque
Table	Ensemble d'enregistrements de même structure	livres
Champ (colonne)	Une information, avec un type	titre VARCHAR(100), annee INT
Enregistrement (ligne)	Un élément de la table	« Le Petit Prince », 1943
Clé primaire	Champ qui identifie chaque ligne de façon unique	id, en AUTO_INCREMENT : numéroté automatiquement

P13.2 La base « bibliotheque »

Dans phpMyAdmin (http://localhost/phpmyadmin), créez la base puis la table, ou collez ce script dans l'onglet SQL. Le script complet du cours, qui crée aussi la base « boutique » des exercices, est fourni par le formateur.

```
CREATE DATABASE bibliotheque CHARACTER SET utf8mb4;
USE bibliotheque;
CREATE TABLE livres (
  id INT AUTO_INCREMENT PRIMARY KEY,
  titre VARCHAR(100) NOT NULL,
  auteur VARCHAR(80) NOT NULL,
  annee INT,
  genre VARCHAR(30),
  disponible TINYINT(1) DEFAULT 1
);
INSERT INTO livres (titre, auteur, annee, genre, disponible) VALUES
('Le Petit Prince', 'Antoine de Saint-Exupéry', 1943, 'Roman', 1),
('Les Misérables', 'Victor Hugo', 1862, 'Roman', 1),
('Le Rouge et le Noir', 'Stendhal', 1830, 'Roman', 0),
('Les Fables', 'Jean de La Fontaine', 1668, 'Poésie', 1),
('Le Bourgeois gentilhomme', 'Molière', 1670, 'Théâtre', 1),
('L''Étranger', 'Albert Camus', 1942, 'Roman', 0),
('Le Prophète', 'Khalil Gibran', 1923, 'Poésie', 1),
('Le Malade imaginaire', 'Molière', 1673, 'Théâtre', 1);
```

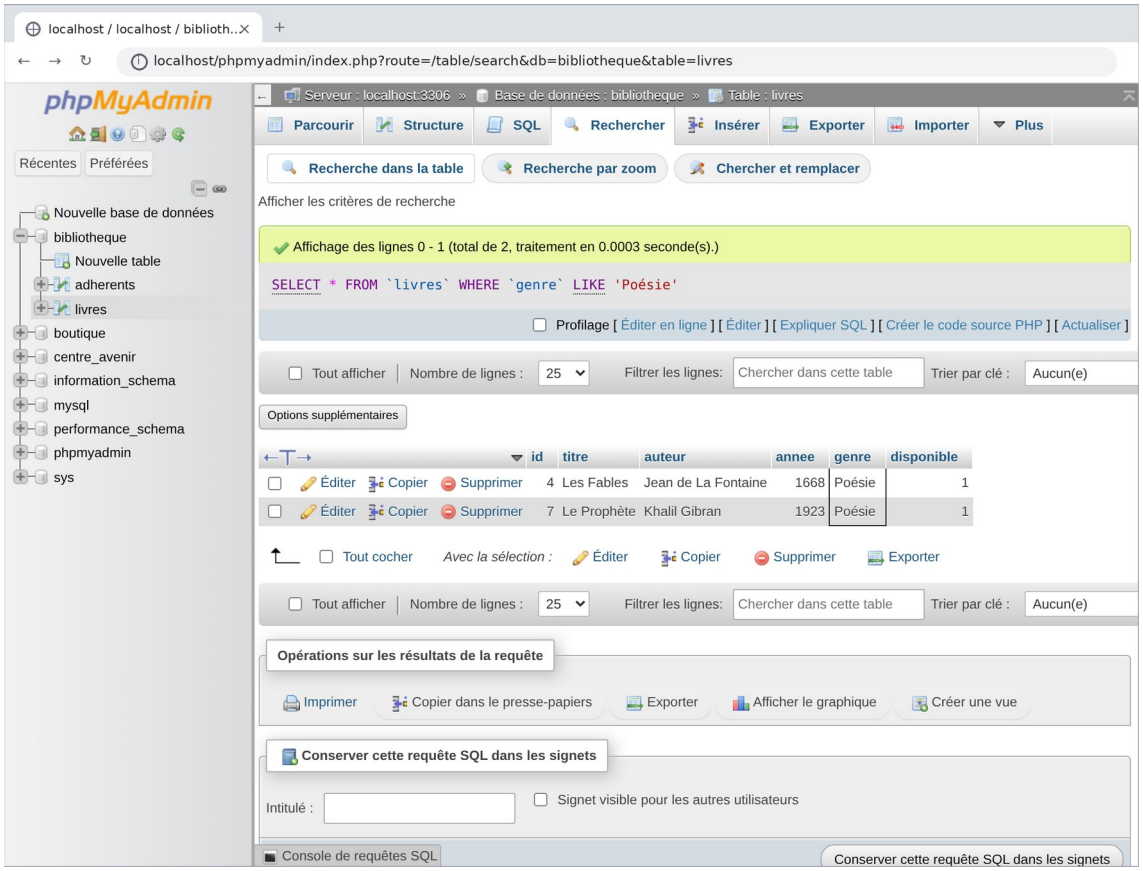
La table « adherents » (nom, prenom, email, ville) complète la base ; elle sert à l'exercice de synthèse du chapitre P20.



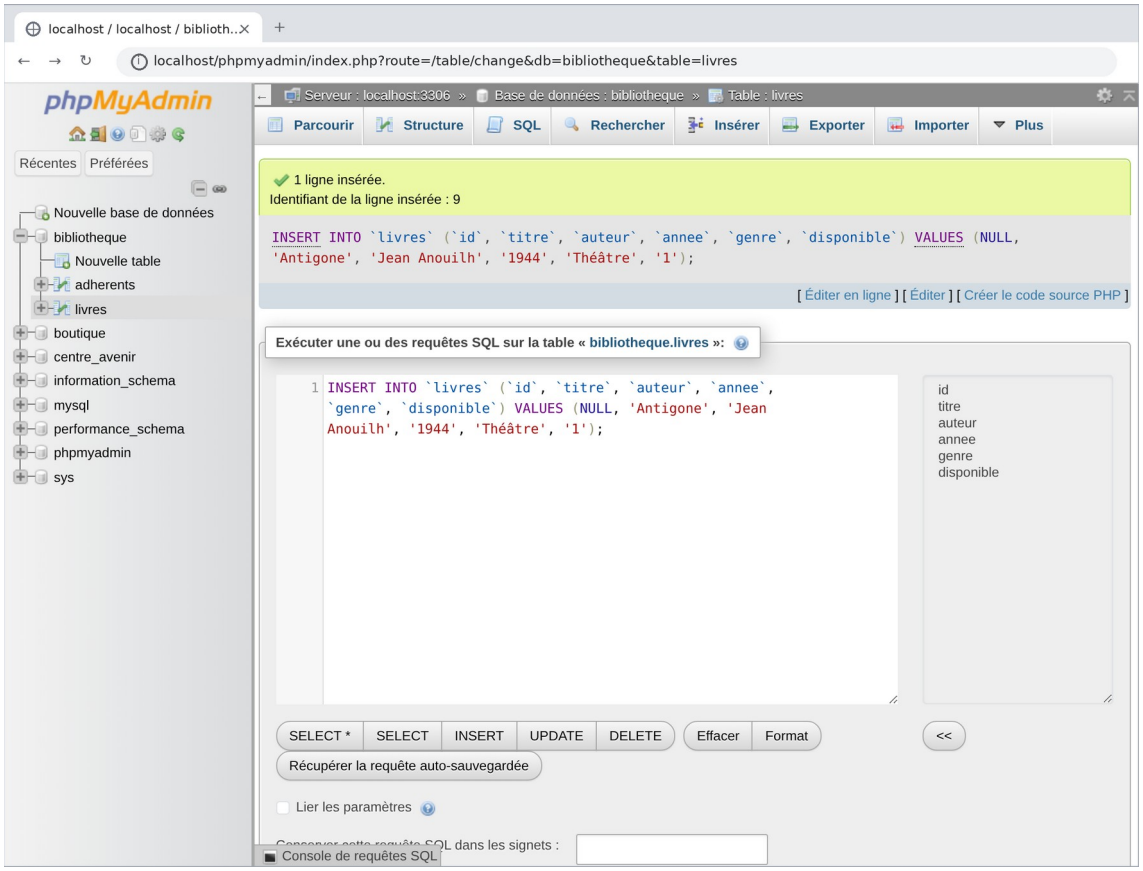
Structure de la table livres dans phpMyAdmin

P13.3 Les requêtes générées en mode assisté

phpMyAdmin construit les requêtes SQL à votre place : vous remplissez un formulaire (onglets Rechercher, Insérer…), il exécute la requête et l'affiche. Il suffit ensuite de la copier dans le script PHP et d'y remplacer les valeurs par des variables.



Onglet Rechercher : genre = Poésie ; phpMyAdmin affiche la requête SELECT générée



Onglet Insérer : la requête INSERT générée apparaît après l'exécution

Requête générée par phpMyAdmin	Version utilisée dans PHP
SELECT * FROM `livres` WHERE `genre` LIKE 'Poésie'	$sql = "SELECT * FROM livres WHERE genre = '$genre'";
INSERT INTO `livres` (`id`, `titre`, ...) VALUES (NULL, 'Antigone', ...)	$sql = "INSERT INTO livres (titre, ...) VALUES ('$titre', ...)";

ASTUCE

Les accents graves (`livres`) entourent les noms de tables et de champs : ils sont facultatifs quand ces noms ne contiennent ni espace ni accent. Les valeurs texte sont entre apostrophes, les nombres sans apostrophes.

Synthèse

À RETENIR

- Base → tables → champs typés ; clé primaire id en AUTO_INCREMENT.
- phpMyAdmin crée la base et génère les requêtes en mode assisté.
- Copier la requête générée, puis remplacer les valeurs par des variables PHP.

Quiz

1. À quoi sert AUTO_INCREMENT sur la clé primaire ?

a) À trier la table
b) À numéroter automatiquement chaque nouvelle ligne
c) À chiffrer les données
d) À interdire les doublons de titre

2. Dans une requête SQL, une valeur texte s'écrit :

a) Sans délimiteur
b) Entre apostrophes
c) Entre accents graves
d) Entre crochets

3. Où trouver la requête générée par phpMyAdmin ?

a) Dans un fichier
b) Affichée au-dessus du résultat après exécution
c) Nulle part
d) Dans la console du navigateur

Exercices

Exercice 1 — Créer la base boutique

Dans phpMyAdmin, créez la base « boutique » et la table « produits » : id (INT, clé primaire, AUTO_INCREMENT), nom (VARCHAR 80), categorie (VARCHAR 40), prix (DECIMAL 8,3), stock (INT). Insérez cinq produits avec l'onglet Insérer.

Exercice 2 — Requêtes en mode assisté

Avec les onglets de phpMyAdmin, sans écrire de SQL : a) affichez les produits de la catégorie « Stockage » ; b) ajoutez un produit ; c) modifiez le stock d'un produit ; d) supprimez un produit. Pour chaque action, recopiez la requête générée.

Corrigés

Quiz

1. b) Le SGBD attribue le numéro suivant.

2. b) 'Poésie'

3. b) phpMyAdmin affiche la requête exécutée.

Exercice 1 — Créer la base boutique

```
CREATE DATABASE boutique CHARACTER SET utf8mb4;
USE boutique;
CREATE TABLE produits (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nom VARCHAR(80) NOT NULL,
  categorie VARCHAR(40),
  prix DECIMAL(8,3),
  stock INT DEFAULT 0
);
INSERT INTO produits (nom, categorie, prix, stock) VALUES
('Clavier sans fil', 'Accessoires', 45.000, 12),
('Souris optique', 'Accessoires', 18.500, 30),
('Écran 24 pouces', 'Écrans', 420.000, 5),
('Clé USB 64 Go', 'Stockage', 29.900, 0),
('Disque SSD 512 Go', 'Stockage', 185.000, 8);
```

Exercice 2 — Requêtes en mode assisté

a) SELECT * FROM `produits` WHERE `categorie` = 'Stockage' ; b) INSERT INTO `produits` (`id`, `nom`, `categorie`, `prix`, `stock`) VALUES (NULL, …) ; c) UPDATE `produits` SET `stock` = … WHERE `produits`.`id` = … ; d) DELETE FROM `produits` WHERE `produits`.`id` = … . Les requêtes UPDATE et DELETE générées contiennent toujours une clause WHERE sur la clé primaire.

Chapitre P14 — Se connecter et sélectionner la base

OBJECTIFS DU CHAPITRE

- Se connecter au serveur MySQL avec mysqli_connect.
- Sélectionner la base de données.
- Arrêter le script avec die en cas d'échec.
- Centraliser la connexion dans un fichier importé avec require.

P14.1 mysqli_connect

```
$connexion = mysqli_connect("localhost", "root", "", "bibliotheque");
//                          serveur      utilisateur  mot de passe  base
```

- Avec XAMPP : serveur localhost, utilisateur root, mot de passe vide.
- Le 4e paramètre sélectionne la base. On peut aussi la sélectionner ensuite avec mysqli_select_db($connexion, "bibliotheque") (fonction hors annexe).
- mysqli_connect retourne la connexion, ou false en cas d'échec : « or die(...) » arrête alors le script avec un message.
- mysqli_close($connexion) ferme la connexion ; elle est fermée automatiquement à la fin du script.

P14.2 Le fichier connexion.php

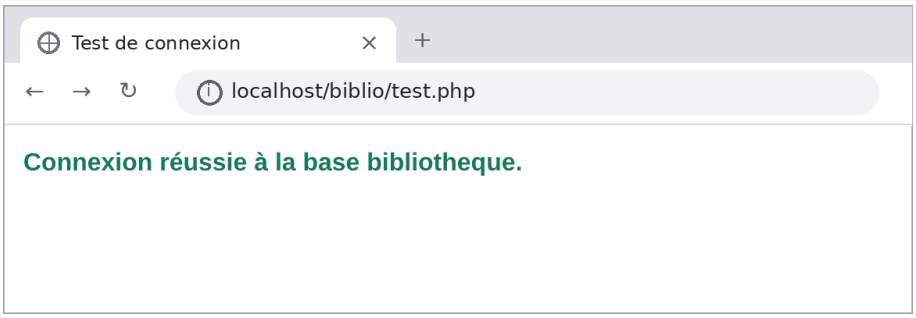
```
<?php
// connexion.php : connexion au serveur et sélection de la base
mysqli_report(MYSQLI_REPORT_OFF);   // erreurs gérées par die et mysqli_error
$connexion = mysqli_connect("localhost", "root", "", "bibliotheque")
    or die("Connexion au serveur impossible");
?>
```

ATTENTION

Depuis PHP 8.1, une erreur MySQL arrête le script par défaut avec un message technique, avant que die ou mysqli_error puissent agir. La ligne mysqli_report(MYSQLI_REPORT_OFF); (hors annexe) rétablit le fonctionnement décrit dans ce cours : écrivez-la toujours en tête de connexion.php.

test.php

```
<?php
$titre = "Test de connexion";
require "entete.php";
require "connexion.php";
echo "<p class='ok'>Connexion réussie à la base bibliotheque.</p>";
mysqli_close($connexion);
require "pied.php";
?>
```

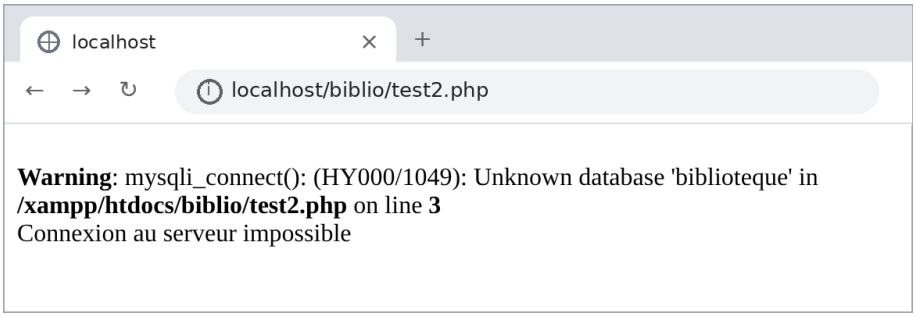


La connexion fonctionne

Si le nom de la base est faux (par exemple « biblioteque »), mysqli_connect échoue : PHP affiche un avertissement qui donne la cause (« Unknown database »), puis die affiche son message et arrête le script.

test2.php

```
<?php
mysqli_report(MYSQLI_REPORT_OFF);
$connexion = mysqli_connect("localhost", "root", "", "biblioteque")
    or die("Connexion au serveur impossible");
echo "Connexion réussie";
?>
```



Base inexistante : le script s'arrête sur le message de die

Synthèse

À RETENIR

- mysqli_connect("localhost", "root", "", "base") or die("...");
- mysqli_report(MYSQLI_REPORT_OFF); en tête de connexion.php.
- connexion.php importé avec require dans chaque page ; mysqli_close pour fermer.

Quiz

1. Avec XAMPP, quel est le mot de passe par défaut de root ?

a) root

b) admin

c) Vide

d) xampp

2. Que fait or die("...") après mysqli_connect ?

a) Ferme la connexion

b) Arrête le script si la connexion échoue

c) Recommence la connexion

d) Supprime la base

3. Pourquoi un fichier connexion.php ?

a) C'est obligatoire

b) Pour écrire la connexion une seule fois et l'importer partout

c) Pour accélérer MySQL

d) Pour cacher la base

Exercices

Exercice 1 — Le fichier de connexion

Créez dans htdocs\boutique le fichier connexion.php vers la base « boutique », puis test.php qui l'importe et affiche « Connexion réussie ». Modifiez ensuite volontairement le nom de la base pour observer le message d'erreur, puis rétablissez-le.

Corrigés

Quiz

1. c) Chaîne vide "".

2. b) die affiche le message et termine le script.

3. b) Une seule modification suffit si les paramètres changent.

Exercice 1 — Le fichier de connexion

```
<?php
// connexion.php : connexion au serveur et sélection de la base
mysqli_report(MYSQLI_REPORT_OFF);   // erreurs gérées par die et mysqli_error
$connexion = mysqli_connect("localhost", "root", "", "boutique")
    or die("Connexion au serveur impossible");
?>
```

```
<?php
require "connexion.php";
echo "Connexion réussie";
?>
```

Chapitre P15 — Afficher des données : SELECT

OBJECTIFS DU CHAPITRE

- Exécuter une requête SELECT avec mysqli_query.
- Parcourir le résultat avec mysqli_fetch_array et une boucle while.
- Compter les lignes avec mysqli_num_rows.
- Afficher le résultat dans un tableau HTML et une fiche détaillée.

P15.1 Les étapes

1. Écrire la requête dans une variable : $sql = "SELECT … FROM … ";
2. L'exécuter : $resultat = mysqli_query($connexion, $sql);
3. Lire les lignes une à une : while ($ligne = mysqli_fetch_array($resultat)) { … }
4. Utiliser chaque champ : $ligne["titre"] (par nom) ou $ligne[1] (par indice).

Fonction	Retourne
mysqli_query($connexion, $sql)	Le résultat de la requête, ou false en cas d'erreur
mysqli_fetch_array($resultat)	La ligne suivante, en tableau associatif et indicé ; false quand il n'y en a plus
mysqli_fetch_row($resultat)	La ligne suivante, en tableau indicé seulement
mysqli_num_rows($resultat)	Le nombre de lignes du résultat
mysqli_error($connexion)	Le message d'erreur de la dernière requête

P15.2 Exemple : le catalogue

catalogue.php

```
<?php
$titre = "Catalogue";
require "entete.php";
require "connexion.php";
$sql = "SELECT id, titre, auteur, annee, disponible FROM livres ORDER BY titre";
$resultat = mysqli_query($connexion, $sql) or die(mysqli_error($connexion));
echo "<h1>Catalogue (" . mysqli_num_rows($resultat) . " livres)</h1>";
echo "<table><tr><th>Titre</th><th>Auteur</th><th>Année</th><th>Disponible</
th></tr>";
while ($ligne = mysqli_fetch_array($resultat)) {
    if ($ligne["disponible"] == 1) { $dispo = "Oui"; } else { $dispo = "Non"; }
    echo "<tr>";
    echo '<td><a href="livre.php?id=' . $ligne["id"] . '">' . $ligne["titre"] .
'</a></td>';
    echo "<td>" . $ligne["auteur"] . "</td><td>" . $ligne["annee"] .
"</td><td>$dispo</td>";
    echo "</tr>";
}
echo "</table>";
mysqli_close($connexion);
require "pied.php";
?>
```



Le catalogue lu dans la table livres

À RETENIR

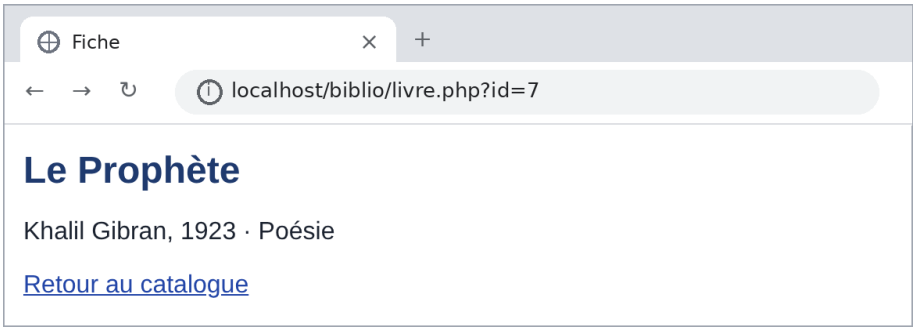
La boucle while s'arrête toute seule : quand il n'y a plus de ligne, mysqli_fetch_array retourne false. « or die(mysqli_error($connexion)) » affiche l'erreur SQL exacte si la requête est mal écrite.

P15.3 Exemple : la fiche d'un livre

Chaque titre du catalogue est un lien vers livre.php?id=… . La fiche lit l'id dans $_GET, le convertit avec (int), puis sélectionne une seule ligne avec WHERE.

livre.php

```
<?php
$titre = "Fiche";
require "entete.php";
require "connexion.php";
if (!isset($_GET["id"])) { die("<p class='erreur'>Aucun livre choisi.</p>"); }
$id = (int) $_GET["id"];
$sql = "SELECT * FROM livres WHERE id = $id";
$resultat = mysqli_query($connexion, $sql);
if (mysqli_num_rows($resultat) == 0) { die("<p class='erreur'>Livre
introuvable.</p>"); }
$livre = mysqli_fetch_array($resultat);
echo "<h1>" . $livre["titre"] . "</h1>";
echo "<p>" . $livre["auteur"] . ", " . $livre["annee"] . " · " .
$livre["genre"] . "</p>";
echo '<p><a href="catalogue.php">Retour au catalogue</a></p>';
require "pied.php";
?>
```



Après un clic sur « Le Prophète » dans le catalogue

Synthèse

À RETENIR

- $resultat = mysqli_query($connexion, $sql) or die(mysqli_error($connexion));
- while ($ligne = mysqli_fetch_array($resultat)) { … $ligne["champ"] … }
- mysqli_num_rows : nombre de lignes ; fiche : WHERE id = (int) $_GET["id"].

Quiz

1. Que retourne mysqli_fetch_array quand il n'y a plus de ligne ?

a) 0

b) false

c) Une chaîne vide

d) Une erreur

2. Comment lire le champ auteur de la ligne courante ?

a) $ligne->auteur

b) $ligne["auteur"]

c) $auteur

d) mysqli_auteur($ligne)

3. Quelle fonction donne le nombre de livres trouvés ?

a) count($sql)

b) mysqli_num_rows($resultat)

c) mysqli_affected_rows

d) strlen($resultat)

Exercices

Exercice 1 — Le catalogue des produits

Dans htdocs\boutique, créez produits.php : affichez tous les produits (nom, catégorie, prix, stock) dans un tableau HTML trié par prix croissant. Affichez « Rupture » en rouge quand le stock vaut 0. Chaque nom est un lien vers produit.php?id=..., qui affiche la fiche du produit.

Exercice 2 — Le nombre de résultats

Avec mysqli_fetch_row, affichez la liste des livres de Molière sous forme de liste à puces (titre et année), précédée de « n livre(s) trouvé(s) ».

Corrigés

Quiz

1. b) La boucle while s'arrête alors.

2. b) La ligne est un tableau associatif.

3. b) Nombre de lignes d'un résultat SELECT.

Exercice 1 — Le catalogue des produits

```
<?php
require "connexion.php";
$resultat = mysqli_query($connexion, "SELECT * FROM produits ORDER BY prix")
    or die(mysqli_error($connexion));
echo "<table><tr><th>Nom</th><th>Catégorie</th><th>Prix</th><th>Stock</th></tr>";
while ($p = mysqli_fetch_array($resultat)) {
    if ($p["stock"] == 0) { $stock = "<span class='erreur'>Rupture</span>"; }
    else { $stock = $p["stock"]; }
    echo '<tr><td><a href="produit.php?id=' . $p["id"] . '">' . $p["nom"] .
'</a></td>';
    echo "<td>" . $p["categorie"] . "</td><td>" . $p["prix"] . "
DT</td><td>$stock</td></tr>";
}
echo "</table>";
?>
```

```php
<?php
// produit.php
require "connexion.php";
if (!isset($_GET["id"])) { die("Aucun produit choisi."); }
$id = (int) $_GET["id"];
$resultat = mysqli_query($connexion, "SELECT * FROM produits WHERE id = $id");
if (mysqli_num_rows($resultat) == 0) { die("Produit introuvable."); }
$p = mysqli_fetch_array($resultat);
echo "<h1>" . $p["nom"] . "</h1><p>" . $p["categorie"] . " · " . $p["prix"] . "
DT · stock " . $p["stock"] . "</p>";
?>
```

Exercice 2 — Le nombre de résultats

```php
<?php
require "connexion.php";
$resultat = mysqli_query($connexion, "SELECT titre, annee FROM livres WHERE
auteur = 'Molière'");
echo "<p>" . mysqli_num_rows($resultat) . " livre(s) trouvé(s)</p><ul>";
while ($ligne = mysqli_fetch_row($resultat)) {
    echo "<li>" . $ligne[0] . " (" . $ligne[1] . ")</li>";
}
echo "</ul>";
?>
```

Avec mysqli_fetch_row, les champs sont lus par indice, dans l'ordre de la requête : 0 pour titre, 1 pour annee.

Chapitre P16 — Rechercher avec un formulaire

OBJECTIFS DU CHAPITRE

- Construire une requête SELECT … WHERE à partir d'un formulaire.
- Remplir une liste déroulante à partir de la base.
- Rechercher un mot dans un texte avec LIKE.
- Gérer le cas « aucun résultat ».

P16.1 Le principe

Le formulaire de recherche utilise la méthode GET : la recherche apparaît dans l'URL et peut être partagée ou ajoutée aux favoris. Le script lit les critères dans $_GET et les place dans la clause WHERE. `LIKE '%mot%'` trouve les textes qui contiennent « mot » (% remplace n'importe quelle suite de caractères).

P16.2 Exemple : rechercher par genre et par auteur

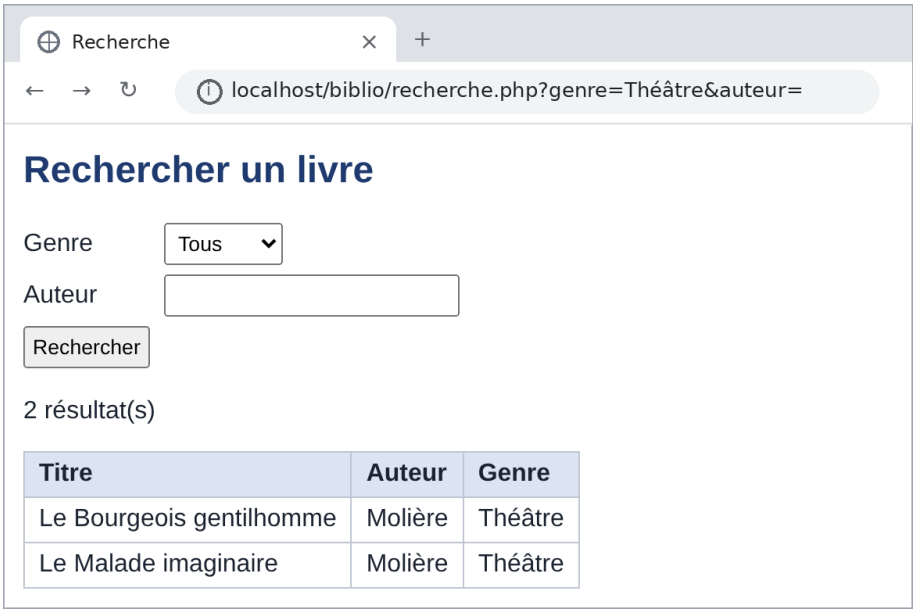
La liste des genres n'est pas écrite à la main : elle est lue dans la table avec SELECT DISTINCT, qui élimine les doublons. Si l'on ajoute un livre d'un nouveau genre, la liste se met à jour seule.

recherche.php

```
<?php
$titre = "Recherche";
require "entete.php";
require "connexion.php";
echo "<h1>Rechercher un livre</h1>";
echo '<form method="get" action="">';
echo '<label for="genre">Genre</label><select name="genre" id="genre"><option
value="">Tous</option>';
$genres = mysqli_query($connexion, "SELECT DISTINCT genre FROM livres ORDER BY
genre");
while ($g = mysqli_fetch_array($genres)) {
    echo "<option>" . $g["genre"] . "</option>";
}
echo '</select><br><label for="auteur">Auteur</label><input type="text"
name="auteur" id="auteur"><br>';
echo '<input type="submit" value="Rechercher"></form>';

if (isset($_GET["genre"])) {
    $genre = $_GET["genre"];
    $auteur = trim($_GET["auteur"]);
    $sql = "SELECT titre, auteur, genre FROM livres WHERE auteur LIKE '%$auteur
%'";
    if ($genre <> "") {
        $sql = $sql . " AND genre = '$genre'";
    }
    $resultat = mysqli_query($connexion, $sql) or die(mysqli_error($connexion));
    $n = mysqli_num_rows($resultat);
    if ($n == 0) {
        echo "<p class='erreur'>Aucun livre trouvé.</p>";
    } else {
        echo "<p>$n
```

```
résultat(s)</p><table><tr><th>Titre</th><th>Auteur</th><th>Genre</th></tr>";
        while ($l = mysqli_fetch_array($resultat)) {
            echo "<tr><td>" . $l["titre"] . "</td><td>" . $l["auteur"] .
"</td><td>" . $l["genre"] . "</td></tr>";
        }
        echo "</table>";
    }
}
require "pied.php";
?>
```



Genre « Théâtre » : la recherche apparaît dans l'URL



Auteur contenant « Hugo », tous genres

ATTENTION

Placer directement une saisie dans une requête expose à l'injection SQL : un utilisateur malveillant peut taper du code SQL dans le champ. Dans ce cours : convertissez toujours les identifiants avec (int), limitez les choix par des listes, et ne publiez jamais ces pages sur Internet sans les protections étudiées plus tard (requêtes préparées).

Synthèse

À RETENIR

- Formulaire de recherche : method="get" ; critères lus dans $_GET.
- Liste déroulante remplie par SELECT DISTINCT ; texte cherché avec LIKE '%mot%'.
- Requête construite par concaténation ; mysqli_num_rows == 0 : « aucun résultat ».

Quiz

1. Que trouve WHERE titre LIKE '%Prince%' ?

a) Les titres égaux à Prince
b) Les titres contenant Prince
c) Les titres commençant par %
d) Rien

2. Pourquoi GET pour un formulaire de recherche ?

a) C'est obligatoire
b) La recherche apparaît dans l'URL et peut être partagée
c) C'est plus sûr
d) POST ne marche pas avec SELECT

3. À quoi sert SELECT DISTINCT genre ?

a) À trier
b) À obtenir chaque genre une seule fois
c) À compter les genres
d) À supprimer des genres

Exercices

Exercice 1 — Recherche par catégorie

Dans la boutique, créez recherche.php : une liste déroulante des catégories (remplie par SELECT DISTINCT) et une case « En stock uniquement ». Affichez les produits correspondants dans un tableau, ou « Aucun produit ».

Exercice 2 — Recherche par mot-clé

Ajoutez à la recherche de la boutique un champ « Mot-clé » qui cherche le texte dans le nom du produit, et deux champs « Prix minimum » et « Prix maximum » (convertis avec (float), ignorés s'ils sont vides).

Corrigés

Quiz

1. b) % remplace n'importe quelle suite de caractères.

2. b) L'URL contient les critères.

3. b) DISTINCT élimine les doublons.

Exercice 1 — Recherche par catégorie

```
<?php
require "connexion.php";
echo '<form method="get" action=""><select name="cat">';
$cats = mysqli_query($connexion, "SELECT DISTINCT categorie FROM produits ORDER
BY categorie");
while ($c = mysqli_fetch_array($cats)) { echo "<option>" . $c["categorie"] .
"</option>"; }
echo '</select> <input type="checkbox" name="stock" id="stock"> <label
for="stock">En stock uniquement</label>';
echo ' <input type="submit" value="Afficher"></form>';
if (isset($_GET["cat"])) {
    $cat = $_GET["cat"];
    $sql = "SELECT nom, prix, stock FROM produits WHERE categorie = '$cat'";
    if (isset($_GET["stock"])) { $sql = $sql . " AND stock > 0"; }
    $res = mysqli_query($connexion, $sql) or die(mysqli_error($connexion));
    if (mysqli_num_rows($res) == 0) { echo "<p>Aucun produit</p>"; }
    else {
        echo "<table><tr><th>Nom</th><th>Prix</th><th>Stock</th></tr>";
        while ($p = mysqli_fetch_array($res)) {
            echo "<tr><td>" . $p["nom"] . "</td><td>" . $p["prix"] .
"</td><td>" . $p["stock"] . "</td></tr>";
        }
        echo "</table>";
    }
}
?>
```

Une case à cocher non cochée n'est pas transmise : isset($_GET["stock"]) suffit à savoir si elle est cochée.

Exercice 2 — Recherche par mot-clé

```
<?php
$mot = trim($_GET["mot"]);
$sql = "SELECT * FROM produits WHERE nom LIKE '%$mot%'";
if ($_GET["min"] <> "") { $sql = $sql . " AND prix >= " . (float) $_GET["min"]; }
if ($_GET["max"] <> "") { $sql = $sql . " AND prix <= " . (float) $_GET["max"]; }
$res = mysqli_query($connexion, $sql) or die(mysqli_error($connexion));
?>
```

Les prix sont convertis avec (float) : la valeur insérée dans la requête est forcément un nombre.

Chapitre P17 — Ajouter des données : INSERT

OBJECTIFS DU CHAPITRE

- Écrire une requête INSERT à partir des données d'un formulaire.
- Contrôler les données avant l'insertion.
- Vérifier le résultat avec mysqli_affected_rows et mysqli_error.
- Gérer les apostrophes dans les textes.

P17.1 La requête INSERT

```
INSERT INTO livres (titre, auteur, annee, genre)
VALUES ('Antigone', 'Jean Anouilh', 1944, 'Théâtre');
```

- On ne donne pas l'id : AUTO_INCREMENT le calcule.
- Textes entre apostrophes, nombres sans apostrophes, dans le même ordre que les champs.
- mysqli_query retourne true si la requête a réussi ; mysqli_affected_rows($connexion) donne le nombre de lignes ajoutées.

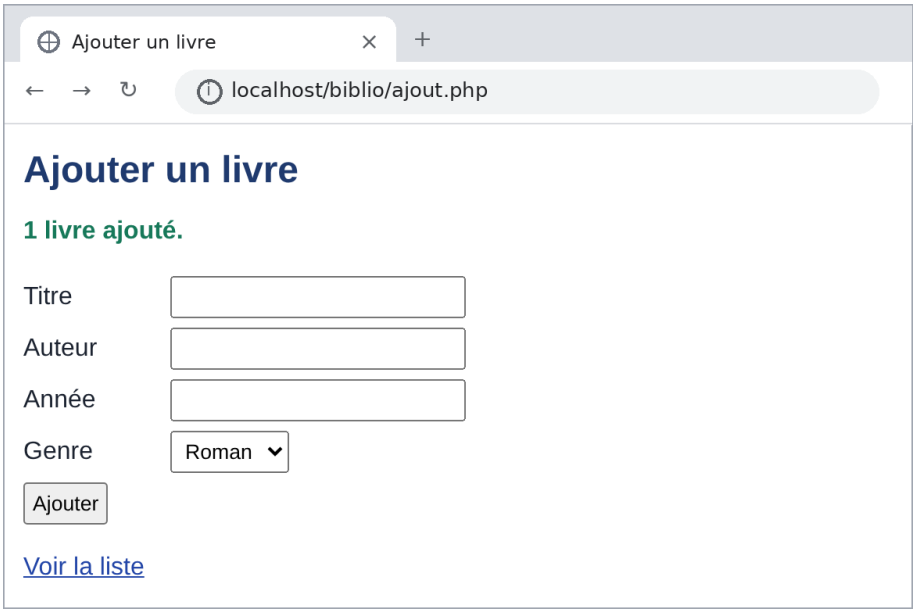
P17.2 Exemple : le formulaire d'ajout

ajout.php

```
<?php
$titre = "Ajouter un livre";
require "entete.php";
require "connexion.php";
echo "<h1>Ajouter un livre</h1>";
if (isset($_POST["ajouter"])) {
    $t = trim($_POST["titre"]);
    $a = trim($_POST["auteur"]);
    $annee = (int) $_POST["annee"];
    $genre = $_POST["genre"];
    if ($t == "" || $a == "") {
        echo "<p class='erreur'>Le titre et l'auteur sont obligatoires.</p>";
    } else {
        $t = str_replace("'", "''", $t);   // une apostrophe est doublée en SQL
        $a = str_replace("'", "''", $a);
        $sql = "INSERT INTO livres (titre, auteur, annee, genre)
                VALUES ('$t', '$a', $annee, '$genre')";
        if (mysqli_query($connexion, $sql)) {
            echo "<p class='ok'>" . mysqli_affected_rows($connexion) . " livre
ajouté.</p>";
        } else {
            echo "<p class='erreur'>Erreur : " . mysqli_error($connexion) .
"</p>";
        }
    }
}
?>
<form method="post" action="">
```

```
  <label for="titre">Titre</label> <input type="text" name="titre"
id="titre"><br>
  <label for="auteur">Auteur</label> <input type="text" name="auteur"
id="auteur"><br>
  <label for="annee">Année</label> <input type="number" name="annee"
id="annee"><br>
  <label for="genre">Genre</label>
  <select name="genre" id="genre">

<option>Roman</option><option>Poésie</option><option>Théâtre</option><option>Essa
i</option>
  </select><br>
  <input type="submit" name="ajouter" value="Ajouter">
</form>
<p><a href="gestion.php">Voir la liste</a></p>
<?php require "pied.php"; ?>
```



Après l'envoi : le livre est enregistré dans la table



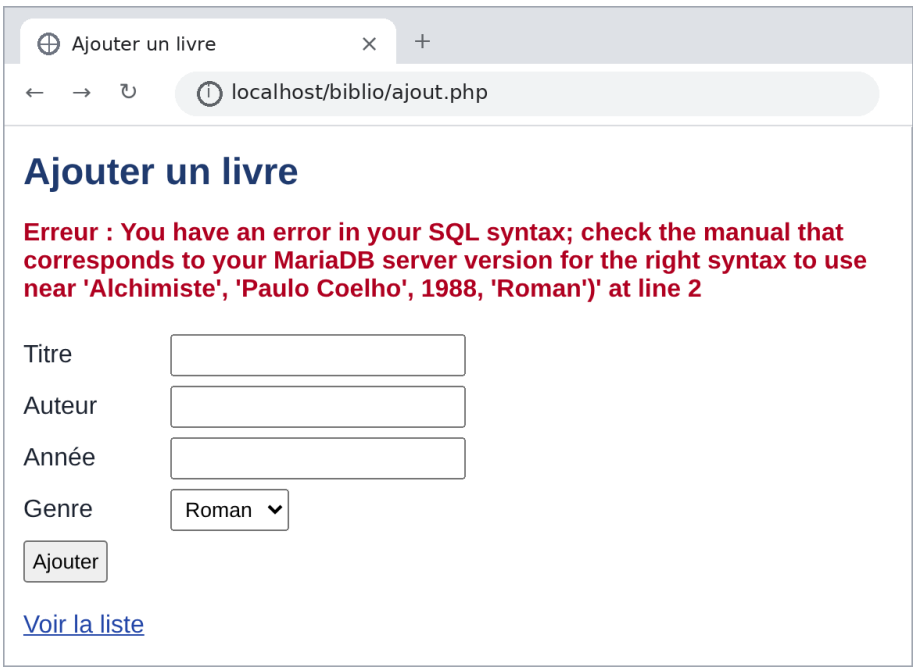
La liste de gestion affiche le livre ajouté (n° 9)

P17.3 Le piège de l'apostrophe

En SQL, l'apostrophe délimite les textes. Le titre « L'Alchimiste » ferme donc le texte trop tôt et la requête devient incorrecte. On double l'apostrophe avec `str_replace("'", "''", $texte)` : c'est l'écriture SQL d'une apostrophe dans un texte. Sans cette ligne, mysqli_error affiche l'erreur :

ajout.php (sans str_replace)

```
$sql = "INSERT INTO livres (titre, auteur, annee, genre)
        VALUES ('$t', '$a', $annee, '$genre')";
// avec $t = "L'Alchimiste" : VALUES ('L'Alchimiste', …) est incorrect
```



Sans str_replace : mysqli_error signale une erreur de syntaxe SQL

Synthèse

À RETENIR

- INSERT INTO table (champs) VALUES (valeurs) ; pas d'id avec AUTO_INCREMENT.
- Contrôler (trim, champs vides, (int)) puis doubler les apostrophes avec str_replace.
- if (mysqli_query(...)) : mysqli_affected_rows ; sinon : mysqli_error.

Quiz

1. Pourquoi ne pas donner l'id dans l'INSERT ?

a) C'est interdit

b) AUTO_INCREMENT le calcule

c) L'id est un texte

d) Pour aller plus vite

2. Que retourne mysqli_affected_rows après un INSERT réussi d'une ligne ?

a) 0

b) 1

c) true

d) L'id

3. Comment écrire L'Étranger dans une requête SQL ?

a) 'L'Étranger'

b) 'L''Étranger'

c) L'Étranger

d) "L''Étranger"

Exercices

Exercice 1 — Ajouter un produit

Dans la boutique, créez ajout.php : nom, catégorie (liste : Accessoires, Écrans, Stockage), prix et stock. Refusez un nom vide ou un prix négatif. Après l'insertion, affichez un message et un lien vers la liste des produits.

Exercice 2 — Inscription d'un adhérent

Dans la bibliothèque, créez adhesion.php : nom, prénom, e-mail et ville (liste remplie par les villes déjà présentes avec SELECT DISTINCT, plus « Autre »). Le formulaire refuse un e-mail sans « @ » (strpos). Après l'ajout, affichez le nombre total d'adhérents.

Corrigés

Quiz

1. b) Le SGBD attribue le numéro suivant.

2. b) Une ligne ajoutée.

3. b) L'apostrophe est doublée.

Exercice 1 — Ajouter un produit

```
<?php
require "connexion.php";
if (isset($_POST["ajouter"])) {
    $nom = str_replace("'", "''", trim($_POST["nom"]));
    $cat = $_POST["categorie"];
    $prix = (float) $_POST["prix"];
    $stock = (int) $_POST["stock"];
    if ($nom == "" || $prix < 0) {
        echo "<p class='erreur'>Nom obligatoire et prix positif.</p>";
    } else {
        $sql = "INSERT INTO produits (nom, categorie, prix, stock) VALUES
('$nom', '$cat', $prix, $stock)";
        if (mysqli_query($connexion, $sql)) {
            echo "<p class='ok'>Produit ajouté.</p><a href='produits.php'>Voir la
liste</a>";
        } else {
            echo "<p class='erreur'>" . mysqli_error($connexion) . "</p>";
        }
    }
}
?>
```

Exercice 2 — Inscription d'un adhérent

```
<?php
require "connexion.php";
if (isset($_POST["inscrire"])) {
    $nom = str_replace("'", "''", trim($_POST["nom"]));
    $prenom = str_replace("'", "''", trim($_POST["prenom"]));
    $email = trim($_POST["email"]);
    $ville = $_POST["ville"];
    if ($nom == "" || $prenom == "" || strpos($email, "@") === false) {
        echo "<p class='erreur'>Saisie incomplète ou e-mail invalide.</p>";
    } else {
        mysqli_query($connexion, "INSERT INTO adherents (nom, prenom, email,
ville)
            VALUES ('$nom', '$prenom', '$email', '$ville')") or
die(mysqli_error($connexion));
        $res = mysqli_query($connexion, "SELECT id FROM adherents");
        echo "<p class='ok'>Bienvenue ! " . mysqli_num_rows($res) . "
adhérents.</p>";
    }
}
?>
```

Chapitre P18 — Supprimer des données : DELETE

OBJECTIFS DU CHAPITRE

- Écrire une requête DELETE limitée par une clause WHERE.
- Transmettre l'identifiant de la ligne par un lien.
- Demander une confirmation avec un formulaire.
- Vérifier la suppression avec mysqli_affected_rows.

P18.1 La requête DELETE

```
DELETE FROM livres WHERE id = 3;
```

ATTENTION

Sans clause WHERE, DELETE FROM livres vide toute la table, sans retour possible. Une suppression se fait toujours sur la clé primaire : WHERE id = $id, avec $id converti par (int).

P18.2 La liste de gestion

La page gestion.php affiche les livres avec, pour chacun, un lien « Modifier » et un lien « Supprimer » qui transmettent l'id par l'URL.

gestion.php

```
<?php
$titre = "Gestion du catalogue";
require "entete.php";
require "connexion.php";
$resultat = mysqli_query($connexion, "SELECT id, titre, auteur, annee, disponible
FROM livres ORDER BY id")
    or die(mysqli_error($connexion));
echo "<h1>Gestion du catalogue</h1>";
echo '<p><a href="ajout.php">+ Ajouter un livre</a></p>';
echo
"<table><tr><th>N°</th><th>Titre</th><th>Auteur</th><th>Dispo.</th><th>Actions</
th></tr>";
while ($l = mysqli_fetch_array($resultat)) {
    if ($l["disponible"] == 1) { $d = "Oui"; } else { $d = "Non"; }
    echo "<tr><td>" . $l["id"] . "</td><td>" . $l["titre"] . "</td><td>" .
$l["auteur"] . "</td><td>$d</td>";
    echo '<td><a href="modifier.php?id=' . $l["id"] . '">Modifier</a> | ';
    echo '<a href="supprimer.php?id=' . $l["id"] . '">Supprimer</a></td></tr>';
}
echo "</table><p>" . mysqli_num_rows($resultat) . " livres</p>";
require "pied.php";
?>
```



Chaque ligne propose Modifier et Supprimer

P18.3 La page de suppression avec confirmation

supprimer.php lit l'id dans $_GET et affiche d'abord une question. La suppression n'a lieu qu'après le clic sur « Oui, supprimer », qui envoie le formulaire en POST à la même adresse (l'id reste dans l'URL).

supprimer.php

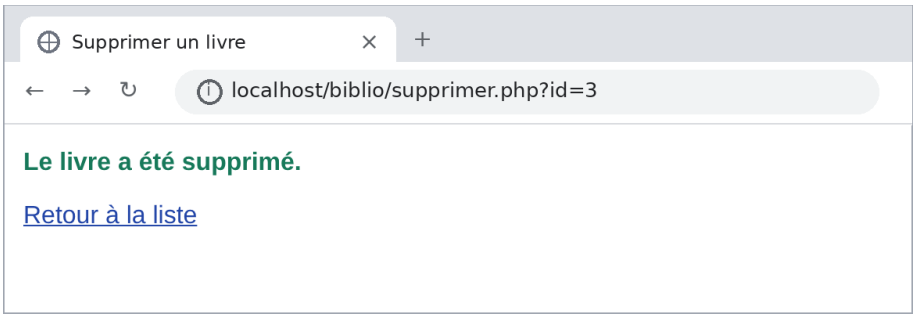
```
<?php
$titre = "Supprimer un livre";
require "entete.php";
require "connexion.php";
if (!isset($_GET["id"])) { die("<p class='erreur'>Aucun livre choisi.</p>"); }
$id = (int) $_GET["id"];

if (isset($_POST["confirmer"])) {
    mysqli_query($connexion, "DELETE FROM livres WHERE id = $id") or
die(mysqli_error($connexion));
    if (mysqli_affected_rows($connexion) == 1) {
        echo "<p class='ok'>Le livre a été supprimé.</p>";
    } else {
        echo "<p class='erreur'>Aucun livre supprimé.</p>";
    }
} else {
    $res = mysqli_query($connexion, "SELECT titre FROM livres WHERE id = $id");
    if (mysqli_num_rows($res) == 0) { die("<p class='erreur'>Livre
introuvable.</p>"); }
    $livre = mysqli_fetch_array($res);
```

```
    echo "<h1>Supprimer « " . $livre["titre"] . " » ?</h1>";
    echo '<form method="post" action="">';
    echo '<input type="submit" name="confirmer" value="Oui, supprimer"> ';
    echo '<a href="gestion.php">Non, annuler</a></form>';
}
echo '<p><a href="gestion.php">Retour à la liste</a></p>';
require "pied.php";
?>
```



Étape 1 : la confirmation est demandée



Étape 2 : après « Oui, supprimer »

Synthèse

À RETENIR

- DELETE FROM table WHERE id = $id ; jamais sans WHERE.
- Lien supprimer.php?id=… ; id converti avec (int).
- Confirmation par un formulaire POST ; résultat vérifié avec mysqli_affected_rows.

Quiz

1. Que fait DELETE FROM produits; ?

a) Supprime la table

b) Supprime toutes les lignes

c) Supprime la dernière ligne

d) Rien sans WHERE

2. Pourquoi demander une confirmation ?

a) C'est obligatoire en SQL

b) Pour éviter une suppression par erreur, définitive

c) Pour accélérer

d) Pour MySQL

3. Après DELETE ... WHERE id = 99 (id inexistant), mysqli_affected_rows vaut :

a) 1

b) 0

c) false

d) Une erreur

Exercices

Exercice 1 — Supprimer un produit

Ajoutez à la liste des produits de la boutique une colonne « Action » avec un lien « Supprimer » vers supprimer.php?id=… . Cette page supprime le produit et affiche « Produit supprimé » ou « Produit introuvable ».

Exercice 2 — Confirmer la suppression

Améliorez la page précédente : elle affiche d'abord le nom et le prix du produit et deux choix, « Oui » (formulaire POST) et « Non » (lien vers la liste). La suppression n'a lieu qu'après « Oui ». Interdisez aussi la suppression d'un produit encore en stock.

Corrigés

Quiz

1. b) Toutes les lignes sont supprimées.

2. b) La suppression est irréversible.

3. b) Aucune ligne supprimée, sans erreur SQL.

Exercice 1 — Supprimer un produit

```
<?php
require "connexion.php";
if (!isset($_GET["id"])) { die("Aucun produit choisi."); }
$id = (int) $_GET["id"];
mysqli_query($connexion, "DELETE FROM produits WHERE id = $id") or
die(mysqli_error($connexion));
if (mysqli_affected_rows($connexion) == 1) { echo "Produit supprimé."; }
else { echo "Produit introuvable."; }
echo " <a href='produits.php'>Retour</a>";
?>
```

Exercice 2 — Confirmer la suppression

```
<?php
require "connexion.php";
$id = (int) $_GET["id"];
$res = mysqli_query($connexion, "SELECT nom, prix, stock FROM produits WHERE id =
$id");
if (mysqli_num_rows($res) == 0) { die("Produit introuvable."); }
$p = mysqli_fetch_array($res);
if ($p["stock"] > 0) { die("Impossible : le produit est encore en stock."); }
if (isset($_POST["oui"])) {
    mysqli_query($connexion, "DELETE FROM produits WHERE id = $id");
    echo "Produit supprimé.";
} else {
    echo "<p>Supprimer " . $p["nom"] . " (" . $p["prix"] . " DT) ?</p>";
    echo '<form method="post" action=""><input type="submit" name="oui"
value="Oui"> <a href="produits.php">Non</a></form>';
}
?>
```

Chapitre P19 — Modifier des données : UPDATE

OBJECTIFS DU CHAPITRE

- Écrire une requête UPDATE limitée par une clause WHERE.
- Pré-remplir un formulaire avec les valeurs de la base.
- Pré-cocher une case et présélectionner une option.
- Enregistrer les modifications et vérifier le résultat.

P19.1 La requête UPDATE

```
UPDATE livres SET annee = 1943, disponible = 1 WHERE id = 1;
```

ATTENTION

Comme DELETE, UPDATE sans WHERE modifie toutes les lignes de la table. Modifiez toujours une ligne précise : WHERE id = $id.

P19.2 Le formulaire pré-rempli

La page modifier.php reçoit l'id par l'URL, lit la ligne avec SELECT et place chaque valeur dans l'attribut value du champ : `value="<?php echo $livre["titre"]; ?>"`. Une case est pré-cochée en écrivant checked ; une option, présélectionnée avec selected.

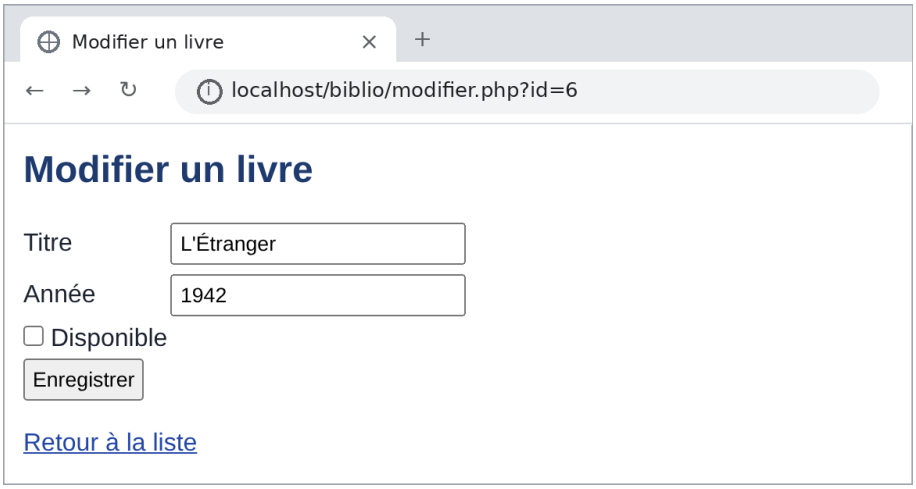
modifier.php

```
<?php
$titre = "Modifier un livre";
require "entete.php";
require "connexion.php";
if (!isset($_GET["id"])) { die("<p class='erreur'>Aucun livre choisi.</p>"); }
$id = (int) $_GET["id"];

if (isset($_POST["enregistrer"])) {
    $t = str_replace("'", "''", trim($_POST["titre"]));
    $annee = (int) $_POST["annee"];
    if (isset($_POST["disponible"])) { $dispo = 1; } else { $dispo = 0; }
    $sql = "UPDATE livres SET titre = '$t', annee = $annee, disponible = $dispo
WHERE id = $id";
    mysqli_query($connexion, $sql) or die(mysqli_error($connexion));
    echo "<p class='ok'>" . mysqli_affected_rows($connexion) . " livre
modifié.</p>";
}

$res = mysqli_query($connexion, "SELECT * FROM livres WHERE id = $id");
if (mysqli_num_rows($res) == 0) { die("<p class='erreur'>Livre
introuvable.</p>"); }
$livre = mysqli_fetch_array($res);
if ($livre["disponible"] == 1) { $coche = "checked"; } else { $coche = ""; }
```

```
?>
<h1>Modifier un livre</h1>
<form method="post" action="">
  <label for="titre">Titre</label>
  <input type="text" name="titre" id="titre" value="<?php echo $livre["titre"]; ?
>"><br>
  <label for="annee">Année</label>
  <input type="number" name="annee" id="annee" value="<?php echo $livre["annee"];
?>"><br>
  <input type="checkbox" name="disponible" id="dispo" <?php echo $coche; ?>>
  <label for="dispo">Disponible</label><br>
  <input type="submit" name="enregistrer" value="Enregistrer">
</form>
<p><a href="gestion.php">Retour à la liste</a></p>
<?php require "pied.php"; ?>
```



Le formulaire reprend les valeurs de la base (case « Disponible » non cochée)



Après l'enregistrement : « Disponible » est mis à jour dans la base

À RETENIR

mysqli_affected_rows vaut 0 si l'on enregistre sans rien changer : MySQL ne compte que les lignes réellement modifiées. Ce n'est pas une erreur.

Synthèse

À RETENIR

- UPDATE table SET champ = valeur, ... WHERE id = $id.
- Formulaire pré-rempli : value="...", checked, selected, à partir d'un SELECT.
- Traitement en tête de page, puis relecture de la ligne pour afficher les nouvelles valeurs.

Quiz

1. Que fait UPDATE produits SET stock = 0; ?

a) Met à 0 le stock du premier produit
b) Met à 0 le stock de tous les produits
c) Rien
d) Une erreur

2. Comment pré-cocher une case ?

a) value="on"
b) checked
c) selected
d) name="coche"

3. Comment présélectionner une option d'une liste ?

a) checked
b) selected
c) value
d) default

Exercices

Exercice 1 — Modifier le stock

Dans la liste des produits, ajoutez à chaque ligne un petit formulaire (méthode POST) avec un champ « Nouveau stock » et un bouton « OK ». Un champ caché transmet l'id. Après l'envoi, la page met à jour le stock puis réaffiche la liste.

Exercice 2 — Modifier un produit

Créez modifier.php?id=... pour la boutique : formulaire pré-rempli (nom, catégorie présélectionnée dans la liste, prix, stock). Après l'enregistrement, affichez le nombre de lignes modifiées.

Corrigés

Quiz

1. b) Sans WHERE, toutes les lignes sont modifiées.

2. b) L'attribut checked.

3. b) <option selected>.

Exercice 1 — Modifier le stock

```
<?php
require "connexion.php";
if (isset($_POST["ok"])) {
    $id = (int) $_POST["id"];
    $stock = (int) $_POST["stock"];
    mysqli_query($connexion, "UPDATE produits SET stock = $stock WHERE id = $id")
or die(mysqli_error($connexion));
}
$res = mysqli_query($connexion, "SELECT * FROM produits");
echo "<table>";
while ($p = mysqli_fetch_array($res)) {
    echo "<tr><td>" . $p["nom"] . "</td><td>" . $p["stock"] . "</td><td>";
    echo '<form method="post" action=""><input type="hidden" name="id" value="' .
$p["id"] . '">';
    echo '<input type="number" name="stock"> <input type="submit" name="ok"
value="OK"></form></td></tr>';
}
echo "</table>";
?>
```

Le champ <input type="hidden"> transmet l'id sans l'afficher.

Exercice 2 — Modifier un produit

```
<?php
// extrait : liste des catégories avec la catégorie actuelle présélectionnée
$cats = ["Accessoires", "Écrans", "Stockage"];
echo '<select name="categorie">';
for ($i = 0; $i < count($cats); $i++) {
    if ($cats[$i] == $p["categorie"]) { $sel = "selected"; } else { $sel = ""; }
    echo "<option $sel>" . $cats[$i] . "</option>";
}
echo "</select>";
// traitement
$sql = "UPDATE produits SET nom = '$nom', categorie = '$cat', prix = $prix, stock
= $stock WHERE id = $id";
?>
```

Chapitre P20 — Une application complète

OBJECTIFS DU CHAPITRE

- Organiser une application de gestion en fichiers.
- Relier les pages par des liens et des formulaires.
- Associer chaque action à la bonne requête et à la bonne méthode.

P20.1 L'organisation des fichiers

```
htdocs\biblio\
├── connexion.php   connexion à la base (importé partout)
├── entete.php      début de page commun, titre variable
├── pied.php        fin de page commune
├── style.css
├── gestion.php     liste + liens Ajouter, Modifier, Supprimer   (SELECT)
├── ajout.php       formulaire d'ajout                           (INSERT)
├── modifier.php    formulaire pré-rempli                        (SELECT +
UPDATE)
└── supprimer.php   confirmation                                 (SELECT +
DELETE)
```

P20.2 Récapitulatif

Action	Déclencheur	Données reçues	Requête	Vérification
Lister	Ouverture de gestion.php	—	SELECT	mysqli_num_rows
Rechercher	Formulaire GET	$_GET	SELECT … WHERE	mysqli_num_rows
Ajouter	Formulaire POST	$_POST	INSERT	mysqli_affected_rows
Modifier	Lien ?id= puis formulaire POST	$_GET["id"] + $_POST	UPDATE … WHERE id	mysqli_affected_rows
Supprimer	Lien ?id= puis confirmation POST	$_GET["id"]	DELETE … WHERE id	mysqli_affected_rows

P20.3 Le parcours complet

Les quatre pages des chapitres P17 à P19 forment l'application. Voici la liste après un ajout, une modification et une suppression :



gestion.php : livre 9 ajouté, livre 6 rendu disponible, livre 3 supprimé

ASTUCE

Testez chaque page dans l'ordre du tableau, puis vérifiez le contenu de la table dans phpMyAdmin (onglet Afficher) : la base doit refléter exactement les actions faites dans les pages.

Synthèse

À RETENIR

- Une page par action ; connexion, en-tête et pied importés avec require.
- Lecture : GET ; écriture : POST ; ligne précise : id converti avec (int).
- Toujours vérifier : mysqli_num_rows, mysqli_affected_rows, mysqli_error.

Quiz

1. Quelle page contient SELECT puis UPDATE ?

a) gestion.php
b) ajout.php
c) modifier.php
d) supprimer.php

2. Quelle méthode pour un formulaire qui modifie la base ?

a) GET
b) POST
c) Les deux
d) Aucune

Exercices

Exercice 1 — La gestion des adhérents

Réalisez l'application complète de gestion de la table adherents : liste (avec le nombre d'adhérents et une recherche par ville), ajout, modification et suppression avec confirmation. Réutilisez connexion.php, entete.php et pied.php.

Corrigés

Quiz

1. c) SELECT pour pré-remplir, UPDATE pour enregistrer.

2. b) Les écritures passent par POST.

Exercice 1 — La gestion des adhérents

Éléments de correction : adherents.php (SELECT avec recherche WHERE ville = … si une ville est choisie, liens modifier/supprimer) ; ajout_adherent.php (contrôles, str_replace, INSERT) ; modifier_adherent.php (SELECT pour value="…", UPDATE … WHERE id) ; supprimer_adherent.php (confirmation POST, DELETE … WHERE id). Chaque page vérifie isset, convertit l'id avec (int) et affiche le résultat de mysqli_affected_rows ou mysqli_error.

Projet final — Le site dynamique « Centre Numérique Avenir »

Vous reprenez le site des cours précédents et le reliez à la base « centre_avenir ». Les pages HTML deviennent des pages PHP.

La base de données

Table	Champs
formations	id (clé primaire, AUTO_INCREMENT), titre, domaine, prix, places
inscriptions	id (clé primaire, AUTO_INCREMENT), nom, prenom, email, telephone, id_formation, date_inscription

Cahier des charges

Page	Fonctionnalités	Notions
entete.php, pied.php, connexion.php	Communs à toutes les pages ; date du jour dans le pied de page	require, date
formations.php	Catalogue des formations lu dans la base, avec lien vers la fiche ; recherche par domaine	SELECT, while, formulaire GET
formation.php?id=…	Fiche d'une formation et places restantes	$_GET, (int), WHERE, mysqli_num_rows
inscription.php	Formulaire d'inscription (liste des formations remplie depuis la base), contrôles, enregistrement	$_POST, str_replace, INSERT
admin.php	Liste des inscrits ; modification et suppression avec confirmation	SELECT, UPDATE, DELETE, mysqli_affected_rows

Grille d'évaluation sur 20

Critère	Points
Organisation : fichiers communs importés avec require, code indenté et commenté	2
Connexion à la base centralisée, erreurs gérées (die, mysqli_error)	2
Catalogue et recherche des formations (SELECT, formulaire GET)	3
Fiche d'une formation via $_GET, id contrôlé	2
Inscription : formulaire POST, contrôles, INSERT	4
Administration : modification (formulaire pré-rempli, UPDATE)	4
Administration : suppression avec confirmation (DELETE)	3

Éléments de correction : inscription.php

```
<?php
$titre = "Inscription";
require "entete.php";
require "connexion.php";
if (isset($_POST["inscrire"])) {
    $nom = str_replace("'", "''", trim($_POST["nom"]));
    $prenom = str_replace("'", "''", trim($_POST["prenom"]));
    $email = trim($_POST["email"]);
    $tel = trim($_POST["telephone"]);
    $idf = (int) $_POST["formation"];
    if ($nom == "" || $prenom == "" || strpos($email, "@") === false ||
strlen($tel) <> 8) {
        echo "<p class='erreur'>Vérifiez les champs saisis.</p>";
    } else {
        $date = date("Y-m-d");
        $sql = "INSERT INTO inscriptions (nom, prenom, email, telephone,
id_formation, date_inscription)
                VALUES ('$nom', '$prenom', '$email', '$tel', $idf, '$date')";
        if (mysqli_query($connexion, $sql)) { echo "<p class='ok'>Inscription
enregistrée.</p>"; }
        else { echo "<p class='erreur'>" . mysqli_error($connexion) . "</p>"; }
    }
}
?>
<form method="post" action="">
  <label for="nom">Nom</label> <input type="text" name="nom" id="nom"><br>
  <label for="prenom">Prénom</label> <input type="text" name="prenom"
id="prenom"><br>
  <label for="email">E-mail</label> <input type="text" name="email"
id="email"><br>
  <label for="telephone">Téléphone</label> <input type="text" name="telephone"
id="telephone"><br>
  <label for="formation">Formation</label>
  <select name="formation" id="formation">
<?php
$res = mysqli_query($connexion, "SELECT id, titre FROM formations ORDER BY
titre");
while ($f = mysqli_fetch_array($res)) {
    echo '<option value="' . $f["id"] . '">' . $f["titre"] . '</option>';
}
?>
  </select><br>
  <input type="submit" name="inscrire" value="S'inscrire">
</form>
<?php require "pied.php"; ?>
```

Annexe — Mémo PHP

Ce mémo reprend l'annexe officielle du programme et les compléments utilisés dans le cours (signalés « hors annexe »).

Types, opérateurs et affichage

Élément	Syntaxe
Types	int, float, string, bool, array
Transtypage	(int) (float) (string) (bool) (array)
Arithmétiques	+ - * / % (modulo)
Comparaison	== <> (ou !=) < <= > >= ; === identique (hors annexe)
Logiques	&& (et) || (ou) ! (non)
Affectation, concaténation	= .
Affichage	echo expression1, expression2;

Fonctions

Domaine	Fonctions
Chaînes	chr, ord, strlen, substr, strpos, strcmp, str_replace, strtolower, strtoupper, trim
Mathématiques	abs, sqrt, round, rand
Tableaux	array, count ; parcours foreach (hors annexe)
Date et heure	checkdate, date, time, strtotime ; date_default_timezone_set (hors annexe)
Diverses	die, isset, require
Superglobales	$_GET (paramètres d'URL), $_POST (formulaire en méthode POST)

MySQLi, interface procédurale

Fonction	Rôle
mysqli_connect(serveur, utilisateur, mdp, base)	Connexion au serveur et sélection de la base
mysqli_query($connexion, $sql)	Exécute une requête : résultat (SELECT) ou true / false
mysqli_fetch_array($resultat)	Ligne suivante, tableau associatif et indicé
mysqli_fetch_row($resultat)	Ligne suivante, tableau indicé
mysqli_num_rows($resultat)	Nombre de lignes d'un résultat

Fonction	Rôle
mysqli_affected_rows($connexion)	Lignes ajoutées, modifiées ou supprimées par la dernière requête
mysqli_error($connexion)	Message d'erreur de la dernière requête
mysqli_close($connexion)	Ferme la connexion
mysqli_report(MYSQLI_REPORT_OFF)	Hors annexe : à placer avant la connexion (PHP 8.1 et suivants)
mysqli_select_db($connexion, base)	Hors annexe : sélection de la base après la connexion

Requêtes SQL

```
SELECT champs FROM table WHERE condition ORDER BY champ;
INSERT INTO table (champ1, champ2) VALUES ('texte', 12);
UPDATE table SET champ1 = 'texte', champ2 = 12 WHERE id = 3;
DELETE FROM table WHERE id = 3;
```